UTOPIOS

Terraform avec OpenStack

Infrastructure as Code : provisionner et gérer un cloud OpenStack avec Terraform

HCL | Variables & tfvars | State | Provider OpenStack | Modules | Workspaces

Utopios® Tous droits réservés

Programme

01

Introduction

- 1.1 Objectifs de la formation
- 1.2 Présentation de Terraform
- 1.3 Pourquoi utiliser Terraform avec OpenStack

02

Mise en place de l'environnement

- 2.1 Installation de Terraform
- 2.2 Configuration de Terraform

03

HCL

- 3.1 Présentation de Terraform HCL
- 3.2 Les différentes variables et commandes
- 3.3 Le flux de travail de Terraform
- 3.4 Les dépendances explicites et implicites
- 3.5 Les cycles de vie des ressources
- 3.6 Les expressions conditionnelles et itératives
- 3.7 Les templates et fonctions intégrées
- 3.8 Locals, fichiers tfvars, TF_VAR et priorités
- 3.9 Comprendre et gérer l'état de Terraform

Utopios® Tous droits réservés

Programme

04

OpenStack et Terraform

- 4.0 Présentation d'OpenStack
- 4.1 Provider
- 4.2 Gestion du réseau
- 4.3 Gestion de l'infrastructure

05

Terraform avancé

- 5.1 Terraform Provisioner
- 5.2 Automatiser son workflow avec Terraform
- 5.3 Les modules Terraform
- 5.4 Gestion des outputs Terraform
- 5.5 Terraform et les environnements

06

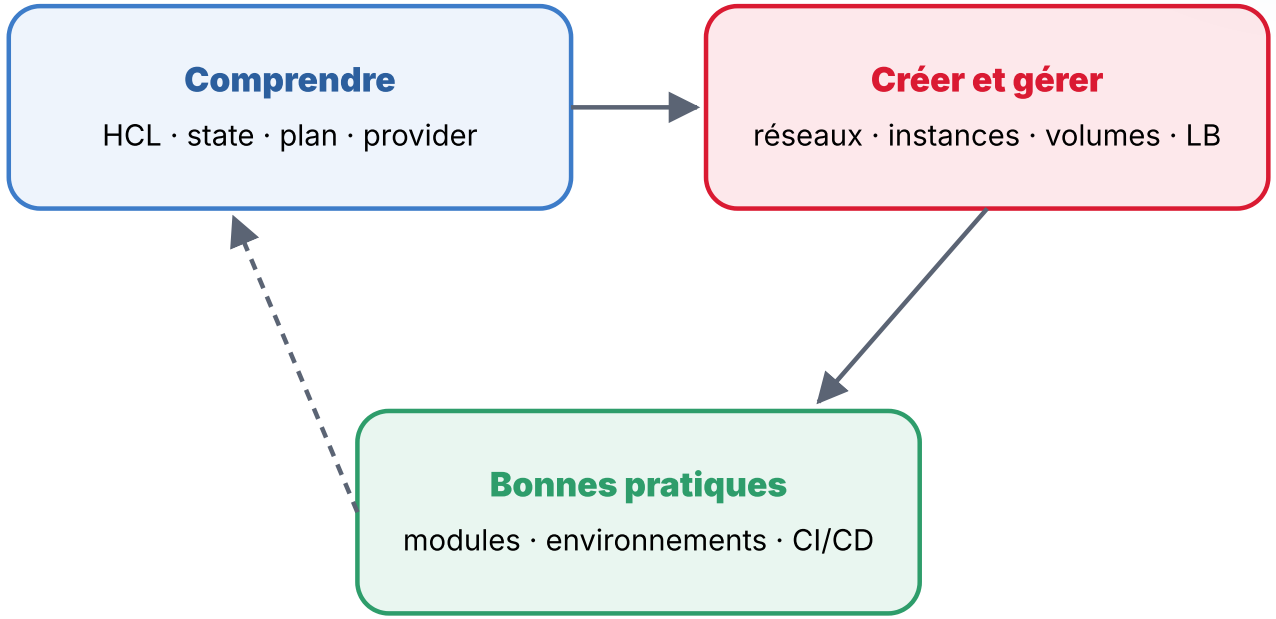
Troubleshooting

- 6.1 Logs et messages d'erreur
- 6.2 Commandes de diagnostic
- 6.3 Versions, dépendances, permissions, quotas

Utopios® Tous droits réservés

Objectifs de la formation

- Comprendre les concepts clés de Terraform
- Apprendre à créer et gérer une infrastructure cloud avec Terraform et OpenStack
- Maîtriser les meilleures pratiques pour utiliser Terraform avec OpenStack



un exercice pratique à chaque étape

Utopios® Tous droits réservés

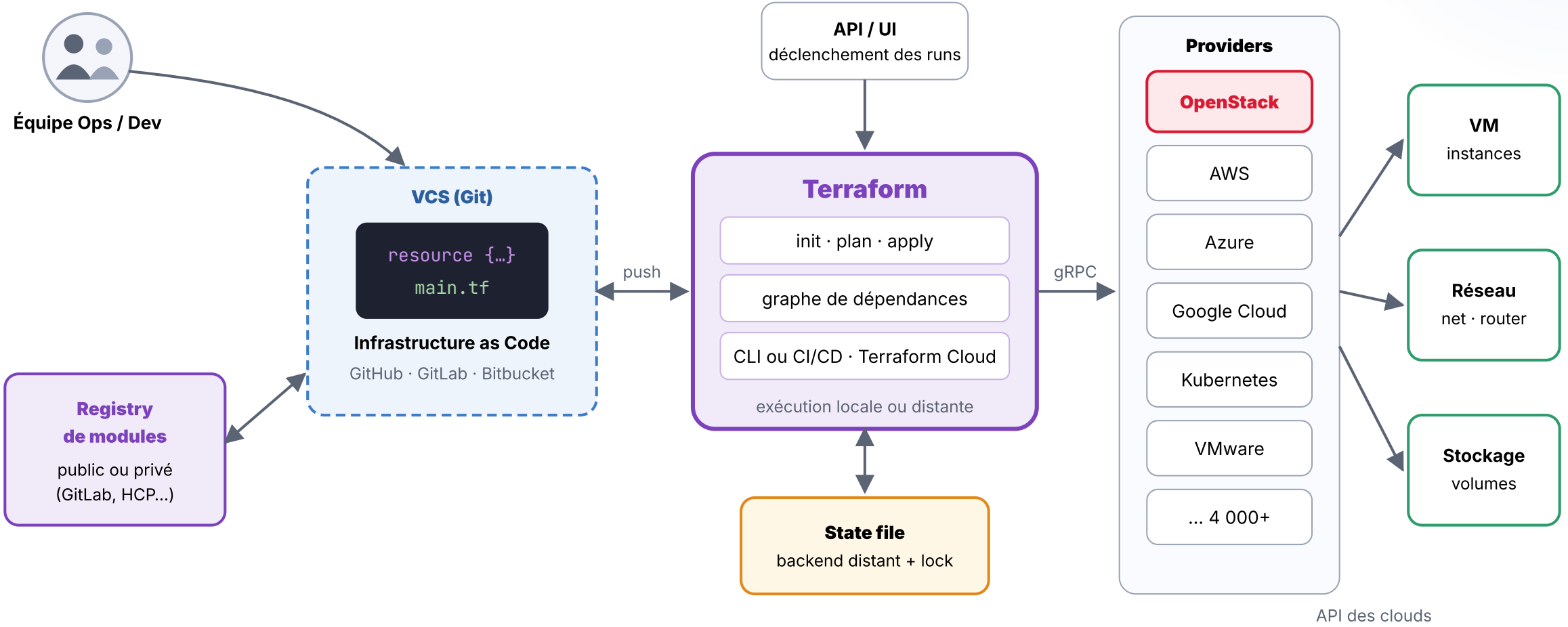
Présentation de Terraform

- **Définition de Terraform :**
  - Terraform est un outil open-source développé par HashiCorp qui permet de définir, provisionner et gérer des ressources d'infrastructure en tant que code (IaC) dans divers fournisseurs de cloud, tels qu'OpenStack.
  - Terraform utilise son propre langage de configuration déclaratif appelé HCL (HashiCorp Configuration Language) pour décrire les ressources souhaitées, puis génère un plan d'exécution pour atteindre l'état souhaité.
- **Avantages de l'Infrastructure as Code (IaC) :**
  - **Versionnage :** Le code de l'infrastructure peut être versionné et contrôlé dans des systèmes de gestion de version (comme Git), permettant un suivi clair des modifications.
  - **Reproductibilité :** Les infrastructures peuvent être déployées de manière cohérente et reproductible, réduisant les erreurs humaines et les différences entre les environnements.
  - **Automatisation :** Le processus de déploiement de l'infrastructure peut être automatisé, réduisant les coûts et les efforts manuels.

Licence : depuis août 2023, Terraform est distribué sous licence BSL 1.1 (source disponible). Son fork open source, **OpenTofu** (Linux Foundation), utilise le même HCL et les mêmes providers.

Utopios® Tous droits réservés

Présentation de Terraform



Utopios® Tous droits réservés

Pourquoi utiliser Terraform avec OpenStack ?

Gestion simplifiée de l'infrastructure

Avec Terraform, vous pouvez définir toute votre infrastructure dans un fichier de configuration. Cela rend la gestion de l'infrastructure plus facile et plus transparente car tout est codé dans un fichier ou un ensemble de fichiers.

Modularité et réutilisabilité

Terraform vous permet de créer des modules pour les différents composants de votre infrastructure. Cela permet de réutiliser les configurations pour différents environnements ou projets.

Gestion des dépendances

Terraform gère les dépendances entre les différents composants de l'infrastructure. Par exemple, si une machine virtuelle dépend d'un réseau virtuel, Terraform s'assurera que le réseau virtuel est créé avant la machine virtuelle.

Gestion des changements

Terraform compare l'état actuel de l'infrastructure avec la configuration définie et ne fait que les modifications nécessaires pour aligner les deux. Cela permet une gestion des changements plus efficace et réduit le risque d'erreurs.

Interopérabilité

Terraform peut être utilisé avec de nombreux autres fournisseurs de services cloud et d'outils de gestion de l'infrastructure. Cela le rend idéal pour les entreprises qui ont une infrastructure hybride ou multi-cloud.

Outil open-source

Terraform est gratuit à utiliser et a une grande communauté de développeurs qui contribuent à son développement et à son support (licence BSL depuis 2023 ; OpenTofu reste open source).

Utopios® Tous droits réservés

Installation de Terraform

1. **Téléchargez Terraform :** Allez sur le site officiel de Terraform et téléchargez la dernière version de Terraform pour votre système d'exploitation : `developer.hashicorp.com/terraform/install`
2. **Décompressez le fichier :** Une fois le fichier téléchargé, décompressez le fichier dans un répertoire de votre choix.
3. **Ajoutez Terraform à votre PATH :**
   - **Sur Windows :** Copiez le fichier `terraform.exe` dans un répertoire qui est dans votre PATH, ou ajoutez le répertoire où se trouve `terraform.exe` à votre PATH.
   - Pour ajouter un répertoire à votre PATH, vous pouvez le faire via les paramètres système ou en utilisant la ligne de commande. Par exemple, si vous avez décompressé `terraform.exe` dans le répertoire `C:\terraform`, vous pouvez exécuter dans l'invite de commande :

```
setx PATH "%PATH%;C:\terraform"
```

Alternative : gestionnaires de paquets `winget install Hashicorp.Terraform` · `choco install terraform` · `brew install hashicorp/tap/terraform` · dépôt APT HashiCorp.

Utopios® Tous droits réservés

Installation de Terraform

- **Sur macOS et Linux :** Déplacez le fichier `terraform` dans un répertoire qui est dans votre PATH. Par exemple, vous pouvez déplacer terraform dans le répertoire `/usr/local/bin` :

```
mv terraform /usr/local/bin/
```

4. **Vérifiez l'installation :** Ouvrez une nouvelle invite de commande ou un terminal et exécutez la commande `terraform -v`. Si Terraform est correctement installé, cette commande affichera la version de Terraform que vous avez installée.

```
$ terraform -v
Terraform v1.x.y
on linux_amd64
```

Utopios® Tous droits réservés

Présentation de Terraform HCL

- HCL (HashiCorp Configuration Language) est un langage de configuration développé par HashiCorp, la société derrière Terraform. HCL est conçu pour être à la fois lisible par l'homme et lisible par la machine, ce qui le rend idéal pour la configuration de l'infrastructure.
- HCL est utilisé pour écrire les fichiers de configuration Terraform. Les fichiers de configuration Terraform décrivent les ressources et les infrastructures à provisionner.
- **Structure de base :** Les fichiers de configuration Terraform, écrits en HCL, sont structurés en trois blocs principaux : provider, resource et output.

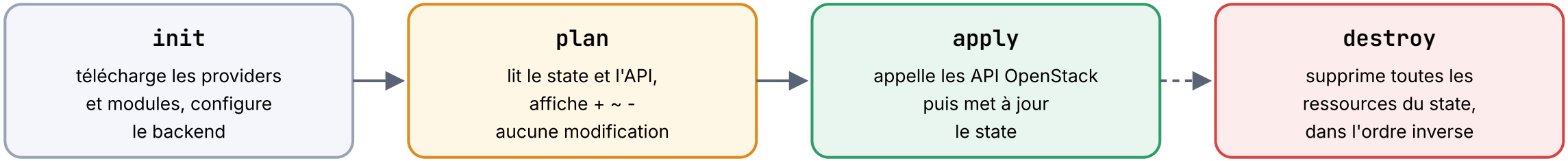


Utopios® Tous droits réservés

Les différentes variables et commandes

Commandes terraform

- `terraform init` : Initialise le répertoire de travail de Terraform. Cette commande doit être exécutée avant d'autres commandes Terraform.
- `terraform plan` : Crée un plan d'exécution. Cette commande montre quelles actions Terraform effectuera pour aligner l'infrastructure sur la configuration.
- `terraform apply` : Applique les changements nécessaires pour aligner l'infrastructure sur la configuration.
- `terraform destroy` : Détruit les ressources créées par Terraform.

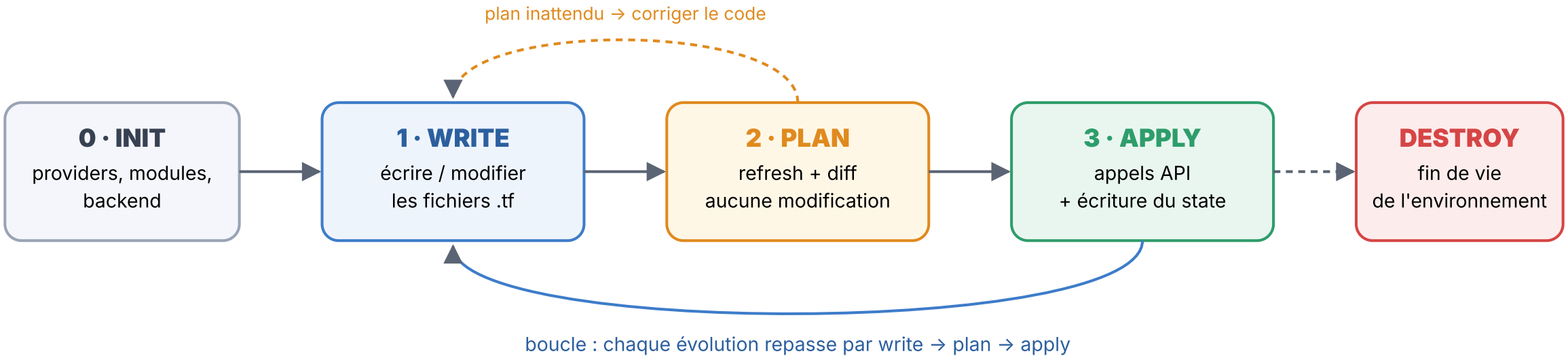


Utiles aussi : `terraform fmt` (formatage) · `terraform validate` (syntaxe) · `terraform output` · `terraform show` · `terraform console`

Utopios® Tous droits réservés

Le flux de travail de Terraform

1. **Write :** La première étape du flux de travail de Terraform consiste à écrire un fichier de configuration. Ce fichier, souvent appelé `main.tf`, définira toutes les ressources que vous souhaitez créer ou gérer. Dans le contexte d'OpenStack, cela signifie définir des ressources comme des machines virtuelles, réseaux... en utilisant le provider openstack et le langage de configuration de Terraform (HCL).
2. **Plan :** Après avoir écrit votre configuration, l'étape suivante est de créer un plan d'exécution avec la commande `terraform plan`. Terraform simule les actions qu'il effectuera en fonction de la configuration actuelle par rapport à l'état actuel de votre infrastructure, sans effectuer réellement de modifications.
3. **Apply :** Une fois que vous avez vérifié que le plan d'exécution est correct, la dernière étape est d'appliquer les modifications avec `terraform apply`. Terraform crée, met à jour ou détruit les ressources nécessaires pour correspondre à la configuration spécifiée dans votre fichier `main.tf`.



Utopios® Tous droits réservés

Les différentes variables et commandes

variables terraform

- Les variables dans Terraform sont un moyen de définir des valeurs qui peuvent être réutilisées dans toute votre configuration. Vous pouvez définir des valeurs par défaut pour vos variables et les substituer au moment de l'exécution si nécessaire.
- **Déclaration :** Vous devez d'abord déclarer vos variables avant de les utiliser. Habituellement, les variables sont déclarées dans un fichier séparé appelé `variables.tf`, mais elles peuvent être déclarées dans n'importe quel fichier `.tf` dans votre configuration.



Utopios® Tous droits réservés

Les différentes variables et commandes

variables terraform

variables.tf

```
variable "datacenter" {
  description = "The datacenter in vSphere to deploy to"
  type        = string
  default     = "dc-01"
}

variable "resource_pools" {
  description = "The resource pools to deploy VMs to"
  type        = list(string)
  default     = ["resource-pool-1", "resource-pool-2"]
}

variable "vm_tags" {
  description = "The tags to apply to virtual machines"
  type        = map(string)
  default     = { Owner = "DevOps", Environment = "Production" }
}
```

Utopios® Tous droits réservés

Les différentes variables et commandes

variables terraform — Utilisation dans main.tf

main.tf

```
output "selected_datacenter" {
  value = var.datacenter
}

output "selected_resource_pools" {
  value = var.resource_pools
}

output "vm_common_tags" {
  value = var.vm_tags
}
```

Utopios® Tous droits réservés

Les différentes variables et commandes

variables terraform

Dans Terraform, les variables peuvent être de plusieurs types, dont voici quelques-uns des plus couramment utilisés.

1. **String :** Il s'agit du type le plus basique. Une variable de type string est simplement une chaîne de caractères.

```
variable "example_string" {
  type    = string
  default = "Hello, World"
}
```

2. **Number :** Ce type est utilisé pour les variables qui sont des nombres. Ils peuvent être des entiers ou des nombres à virgule flottante.

```
variable "example_number" {
  type    = number
  default = 42
}
```

Utopios® Tous droits réservés

Les différentes variables et commandes

variables terraform

3 **Bool :** Ce type est utilisé pour les variables qui sont soit true, soit false.

```
variable "example_bool" {
  type    = bool
  default = true
}
```

4 **List (ou tuple) :** Une liste est une collection ordonnée d'éléments du même type.

```
variable "example_list" {
  type    = list(string)
  default = ["one", "two", "three"]
}
```

Utopios® Tous droits réservés

Les différentes variables et commandes

variables terraform

5 **Map :** Une carte est une collection non ordonnée d'éléments de type clé-valeur.

```
variable "example_map" {
  type = map(string)
  default = {
    key1 = "value1"
    key2 = "value2"
  }
}
```

6 **Set :** Un ensemble est une collection non ordonnée d'éléments uniques du même type.

```
variable "example_set" {
  type    = set(string)
  default = ["one", "two", "three"]
}
```

Utopios® Tous droits réservés

Les différentes variables et commandes

variables terraform

7 **Object :** Un objet est une collection d'attributs avec des types spécifiés.

```
variable "example_object" {
  type = object({
    attribute1 = string
    attribute2 = list(number)
  })
  default = {
    attribute1 = "value"
    attribute2 = [1, 2, 3]
  }
}
```

Utopios® Tous droits réservés

Les différentes variables et commandes EXERCICE

Exercice

Configurer une configuration de base Terraform qui utilise différentes variables pour personnaliser les valeurs des sorties, y compris des variables qui dépendent d'autres variables.

Dans le fichier `variables.tf`, déclarez les variables suivantes :

- Une variable `first_name` de type string sans valeur par défaut.
- Une variable `last_name` de type string sans valeur par défaut.
- Une variable `age` de type number sans valeur par défaut.
- Une variable `is_student` de type bool avec une valeur par défaut de `false`.
- Une variable `courses` de type list(string) avec une valeur par défaut de `["Math", "Science"]`.
- Une variable `grades` de type map(number) avec une valeur par défaut de `{ Math = 90, Science = 85 }`.
- Une variable `student` de type object qui a les attributs suivants : `first_name` de type string, `last_name` de type string, `age` de type number, `is_student` de type bool, `courses` de type list(string), `grades` de type map(number).

Utopios® Tous droits réservés

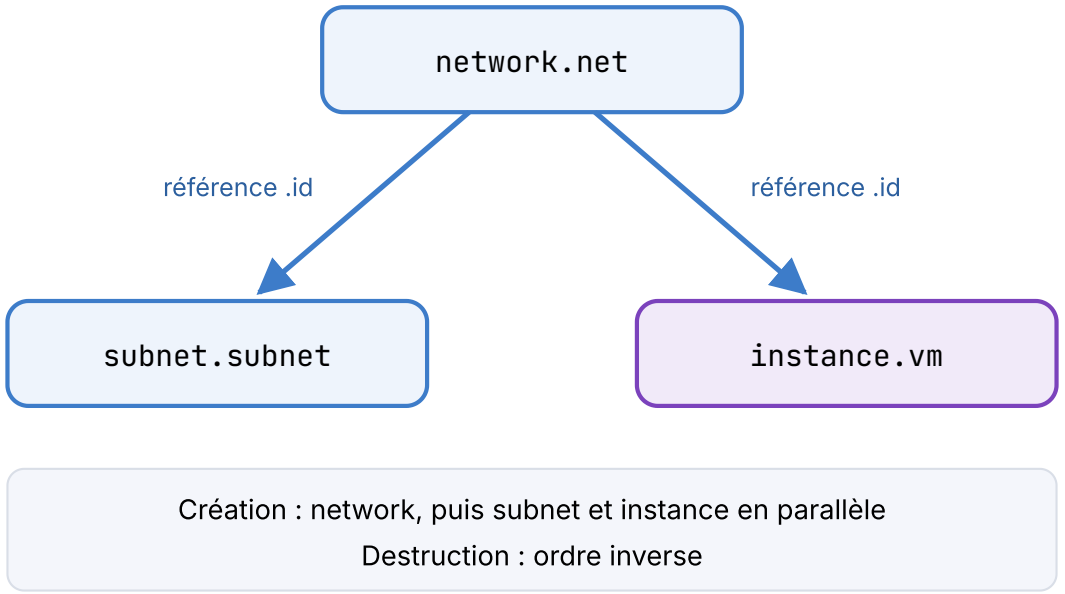
Les dépendances explicites et implicites entre les ressources

- Les dépendances entre les ressources sont essentielles pour déterminer l'ordre dans lequel les ressources doivent être créées, mises à jour ou détruites.
- **1. Dépendances implicites :** Les dépendances implicites sont créées automatiquement par Terraform lorsque vous utilisez une référence à une autre ressource dans la configuration d'une ressource.

```
resource "openstack_networking_network_v2" "net" {
  name = "app-net"
}

resource "openstack_networking_subnet_v2" "subnet" {
  network_id = openstack_networking_network_v2.net.id
  cidr       = "192.168.0.0/24"
}

resource "openstack_compute_instance_v2" "vm" {
  name = "vm-01"
  network {
    uuid = openstack_networking_network_v2.net.id
  }
}
```



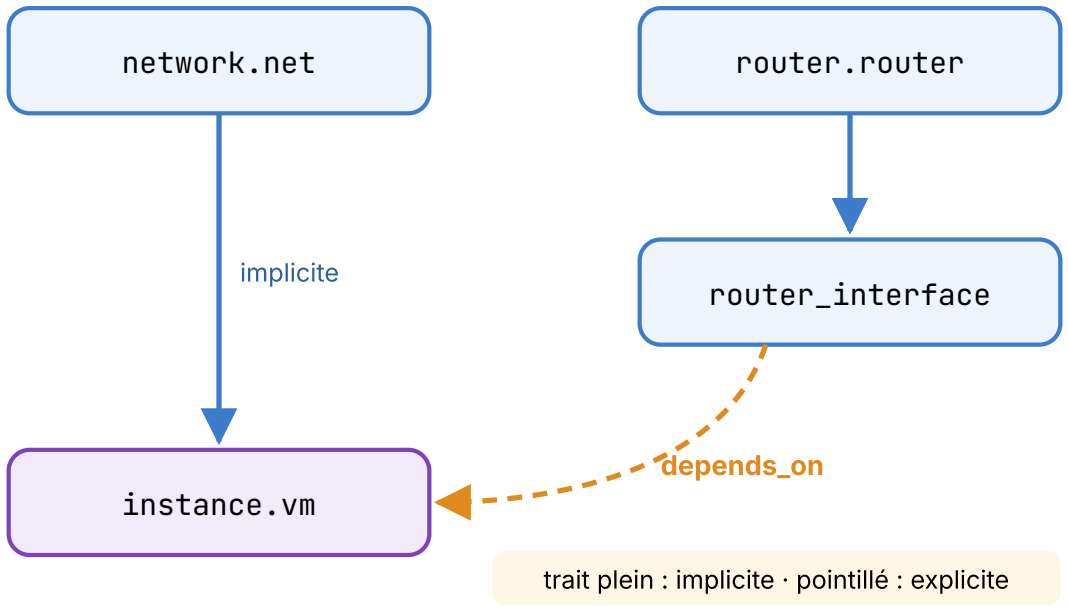
Utopios® Tous droits réservés

Les dépendances explicites et implicites entre les ressources

- **2. Dépendances explicites :** Vous pouvez également spécifier des dépendances explicites à l'aide de l'attribut `depends_on`. Cela peut être nécessaire dans certaines situations où Terraform ne peut pas déterminer les dépendances automatiquement.

```
resource "openstack_networking_router_interface_v2" "itf" {
  router_id = openstack_networking_router_v2.router.id
  subnet_id = openstack_networking_subnet_v2.subnet.id
}

resource "openstack_compute_instance_v2" "vm" {
  name = "vm-01"
  network {
    uuid = openstack_networking_network_v2.net.id
  }
  # la VM a besoin d'Internet au démarrage :
  # aucune référence ne lie la VM au router
  depends_on = [
    openstack_networking_router_interface_v2.itf
  ]
}
```

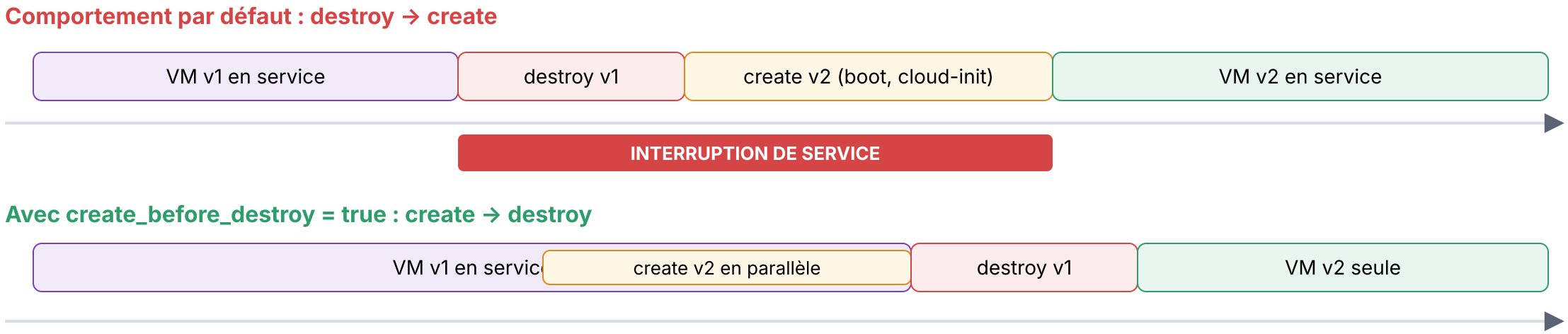


Utopios® Tous droits réservés

Les cycles de vie des ressources

Les cycles de vie des ressources dans Terraform définissent l'ordre et le comportement des actions qui seront effectuées sur une ressource. Par exemple, lorsqu'une ressource doit être créée, mise à jour ou détruite. Le bloc `lifecycle` dans la configuration d'une ressource permet de contrôler ce comportement.

- `create_before_destroy` : Un booléen qui, lorsqu'il est défini sur true, indique à Terraform de créer la nouvelle ressource avant de détruire l'ancienne lors d'une mise à jour.
- `prevent_destroy` : Un booléen qui, lorsqu'il est défini sur true, empêchera la ressource d'être détruite. Cela peut être utile pour éviter la suppression accidentelle de ressources critiques.
- `ignore_changes` : Une liste d'attributs qui seront ignorés lors de la mise à jour d'une ressource. Cela peut être utile pour ignorer les modifications apportées par des sources externes à Terraform.
- `replace_triggered_by` : Il permet de spécifier une liste de références (ressources ou attributs de ressources) qui, lorsqu'elles sont modifiées, déclenchent la destruction et la recréation de la ressource au lieu de simplement la mettre à jour.



Utopios® Tous droits réservés

Les cycles de vie des ressources

Exemple

```
resource "example_resource" "example" {
  # ...

  lifecycle {
    create_before_destroy = true
    prevent_destroy       = true
    ignore_changes        = [example_attribute]
    replace_triggered_by  = [other_resource.example.id]
  }
}
```

create_before_destroy
nouvelle ressource créée → ancienne détruite (pas de coupure)

prevent_destroy
tout plan qui détruit la ressource échoue

ignore_changes
écart sur l'attribut listé = « No changes »

replace_triggered_by
la ressource référencée change → recréation de celle-ci

Utopios® Tous droits réservés

Les cycles de vie des ressources Exercice

1. Déclarez deux variables, `example_attribute1` et `example_attribute2`, toutes deux de type string avec des valeurs par défaut.
2. Configurez quatre ressources fictives : `example_resource_1`, `example_resource_2`, `example_resource_3`, et `example_resource_4`.
   - Pour `example_resource_1`, utilisez la variable `example_attribute1` et configurez son bloc lifecycle pour que la nouvelle ressource soit créée avant la destruction de l'ancienne, et pour ignorer les modifications de `example_attribute1`.
   - Pour `example_resource_2`, utilisez la variable `example_attribute2`, configurez une dépendance explicite sur `example_resource_1` en utilisant `depends_on`, et configurez son bloc lifecycle pour empêcher la destruction de la ressource.
   - Pour `example_resource_3`, configurez une dépendance explicite sur `example_resource_2` et `example_resource_1` en utilisant `depends_on`.
   - Pour `example_resource_4`, configurez une dépendance explicite sur `example_resource_3` en utilisant `depends_on`, et configurez son bloc lifecycle pour ignorer les modifications de toutes les propriétés, à l'exception de `example_attribute2`.

Astuce : les ressources fictives peuvent être des `terraform_data` (attribut `input`), intégrées à Terraform sans provider.

Utopios® Tous droits réservés

Les expressions conditionnelles et itératives

Les expressions conditionnelles dans Terraform permettent d'évaluer une condition et de retourner une valeur en fonction du résultat de cette condition. La syntaxe de base pour une expression conditionnelle est la suivante :

```
condition ? true_value : false_value
```

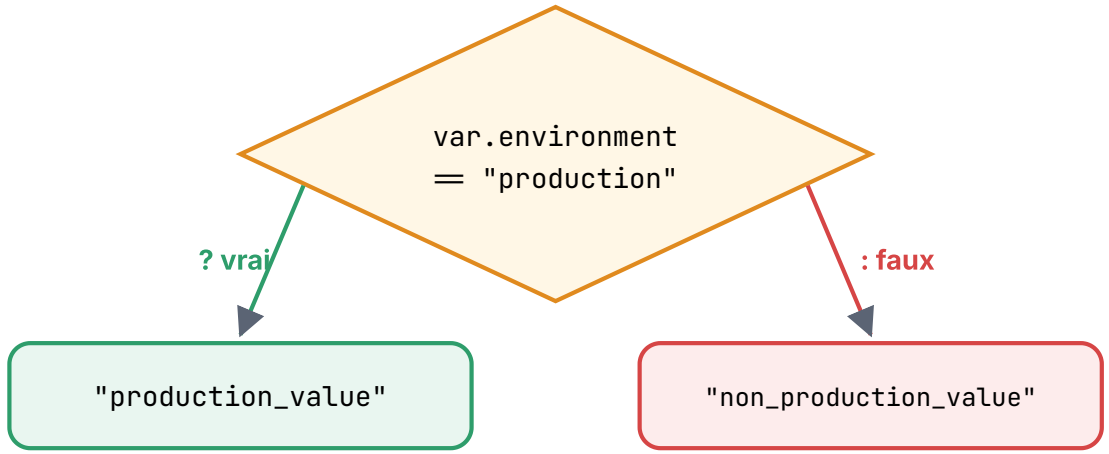
Exemple :

variables.tf

```
variable "environment" {
  description = "The environment"
  type        = string
}
```

main.tf

```
resource "example_resource" "example" {
  example_attribute = (var.environment == "production"
    ? "production_value"
    : "non_production_value")
}
```



Utopios® Tous droits réservés

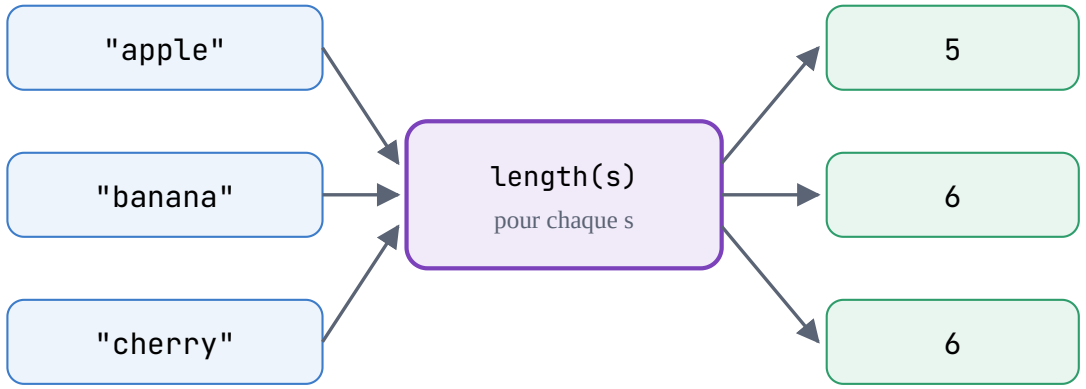
Les expressions conditionnelles et itératives

Les expressions itératives, comme `for` et `for_each`, permettent de créer plusieurs ressources ou d'effectuer des opérations sur chaque élément d'une collection.

1. Expression for : L'expression `for` peut être utilisée pour transformer une liste ou un ensemble de valeurs. Elle peut également être utilisée pour filtrer ou transformer les éléments d'une collection.

```
variable "list_of_strings" {
  description = "A list of strings"
  type        = list(string)
  default     = ["apple", "banana", "cherry"]
}

output "lengths_of_strings" {
  value = [for s in var.list_of_strings : length(s)]
}
```



Utopios® Tous droits réservés

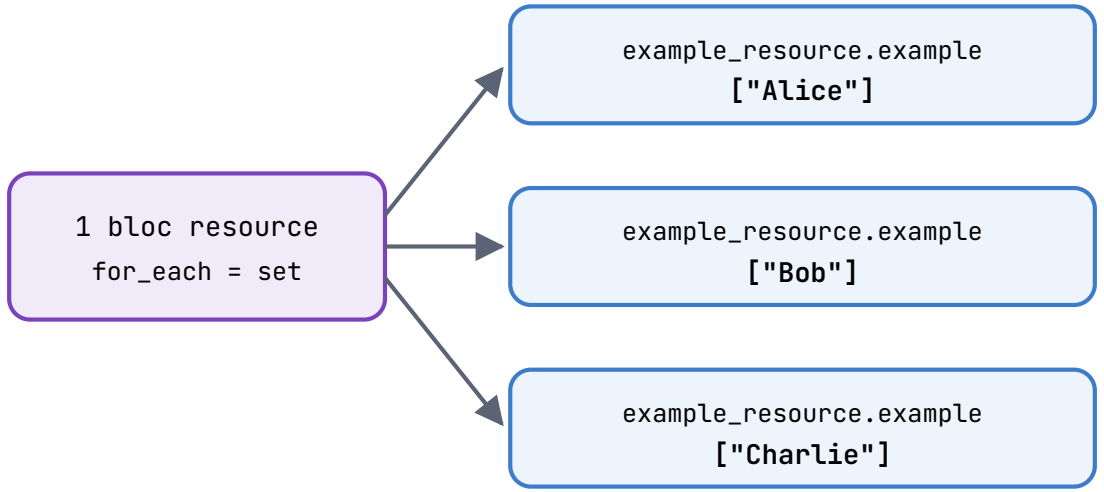
Les expressions conditionnelles et itératives

2. L'attribut for_each : L'attribut `for_each` peut être utilisé pour créer une instance de ressource pour chaque élément dans une collection.

```hcl
variable "set_of_names" {
  description = "A set of names"
  type        = set(string)
  default     = ["Alice", "Bob", "Charlie"]
}

resource "example_resource" "example" {
  for_each = var.set_of_names

  name = each.value
}
```



Chaque instance est identifiée par sa **clé** (`each.key`) : retirer « Bob » ne détruit que l'instance `["Bob"]` .

Utopios® Tous droits réservés

Les expressions conditionnelles et itératives EXERCICE

Exercice

- **Structures conditionnelles**
  - Créez un fichier `main.tf` et déclarez une variable `environment` qui prendra deux valeurs possibles : « production » ou « development ».
  - Déclarez une autre variable `type` qui prendra deux valeurs possibles : « small » ou « large ».
  - Utilisez une expression conditionnelle pour définir une troisième variable `instance_size` basée sur la valeur de `type` :
    - Si type est « small », instance_size devrait être « micro ».
    - Si type est « large », instance_size devrait être « big ».
- **Structures itératives**
  - Déclarez une variable `names` qui est une liste de noms.
  - Utilisez une expression `for` pour créer une liste de messages de bienvenue pour chaque nom dans la liste `names`.
  - Chaque message de bienvenue doit être formaté comme suit : `"Welcome, [name]!"`
  - Affichez la valeur de `instance_size` et la liste des messages de bienvenue.

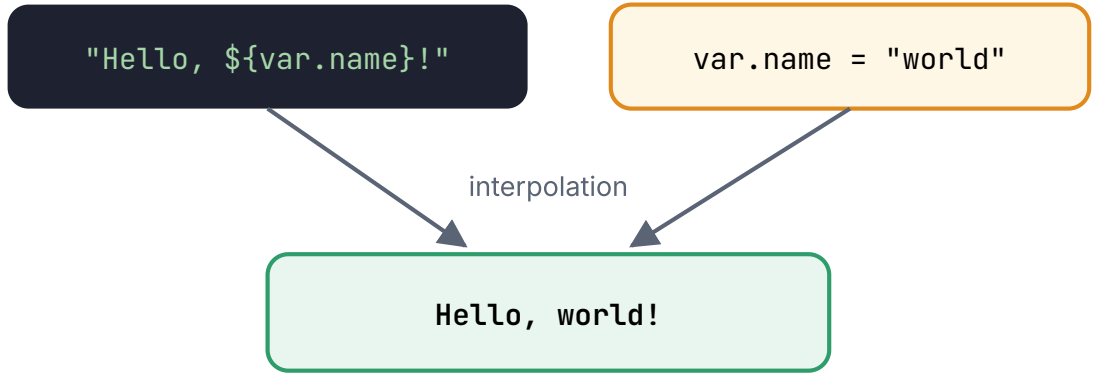
Utopios® Tous droits réservés

Les templates et fonctions intégrées

1. Templates : Les templates dans Terraform sont utilisés pour insérer des valeurs dans des chaînes de caractères. Vous pouvez utiliser les expressions et les fonctions de Terraform dans un template. La syntaxe des templates est `${...}`.

```
variable "name" {
  default = "world"
}

output "greeting" {
  value = "Hello, ${var.name}!"
}
```



Utopios® Tous droits réservés

Les templates et fonctions intégrées

2. Fonctions intégrées : Terraform a un grand nombre de fonctions intégrées qui peuvent être utilisées pour transformer et combiner des valeurs. Ces fonctions peuvent être utilisées dans n'importe quelle expression dans une configuration Terraform.

```
variable "list" {
  default = [1, 2, 3]
}
output "sum" {
  value = sum(var.list)
}
```

Quelques autres fonctions intégrées communes sont :

- `length(list)` : Retourne le nombre d'éléments dans une liste.
- `join(separator, list)` : Combine les éléments d'une liste en une seule chaîne, en insérant un séparateur entre chaque élément.
- `split(separator, string)` : Divise une chaîne en une liste d'éléments, en utilisant un séparateur pour déterminer les limites de chaque élément.
- `templatefile(path, vars)` : Lit un fichier template et remplace ses variables (utile pour l'exercice suivant).

Utopios® Tous droits réservés

Les templates et fonctions intégrées EXERCICE

Exercice

- Créez un fichier `main.tf` et déclarez une variable `names` qui est une liste de noms.
- Combinez les noms de la liste `names` en une seule chaîne séparée par des virgules.
- Comptez le nombre de noms dans la liste `names`.
- Créez un fichier `example.tpl` avec le contenu suivant :

```
Hello, ${name}!
```

- Utilisez le fichier `example.tpl` et remplacez la variable `${name}` par chaque nom de la liste `names`.

Utopios® Tous droits réservés

Les locals

Des valeurs calculées une fois et réutilisées dans toute la configuration

locals.tf

```
locals {
  prefix = "${var.project}-${var.environment}"

  is_prod   = var.environment == "production"
  vm_count  = local.is_prod ? 3 : 1
  flavor    = local.is_prod ? "m1.large" : "m1.small"

  common_tags = ["project:${var.project}", "env:${var.environment}"]
}

resource "openstack_compute_instance_v2" "web" {
  count       = local.vm_count
  name        = "${local.prefix}-web-${count.index + 1}"
  flavor_name = local.flavor
  tags        = local.common_tags
}
```

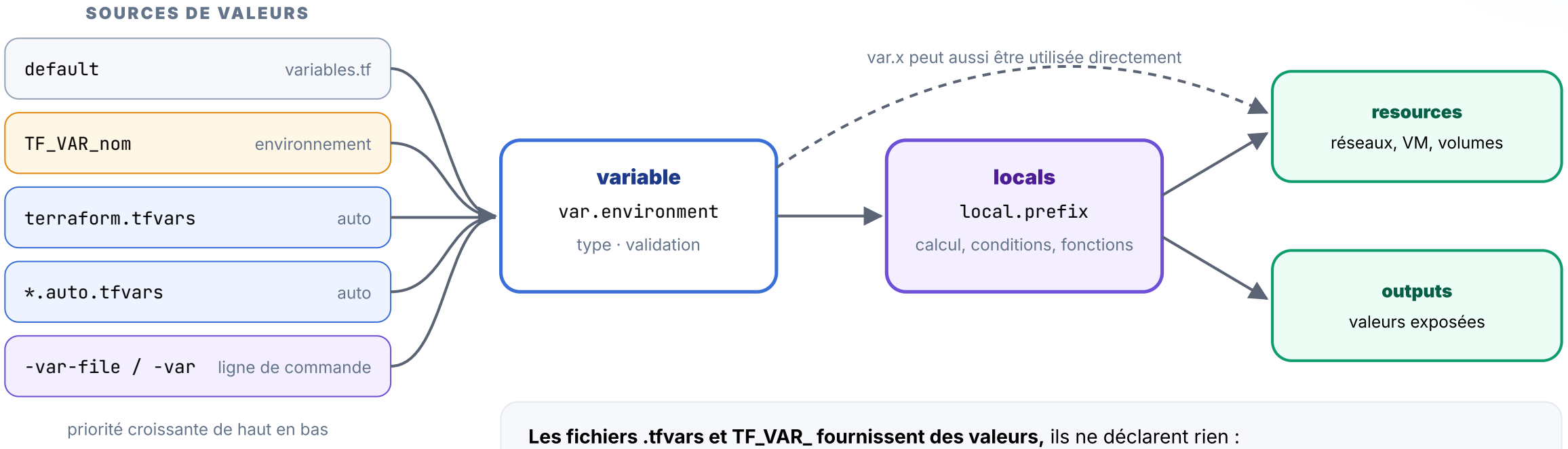
- Déclarés dans un bloc `locals` (pluriel), utilisés avec `local.nom` (singulier)
- Acceptent toute expression : conditions, `for`, fonctions, templates
- Évitent de répéter la même expression dans plusieurs ressources
- Ne peuvent pas être modifiés de l'extérieur, contrairement aux variables

	VARIABLE	LOCAL
Valeur fournie par	l'utilisateur	le code
Contenu	valeur littérale	toute expression
Référence	`var.x`	`local.x`
Usage	ce qui change entre environnements	nommage, tags, valeurs dérivées

Utopios® Tous droits réservés

Variables, locals et valeurs : vue d'ensemble

D'où viennent les valeurs, et comment elles circulent jusqu'aux ressources

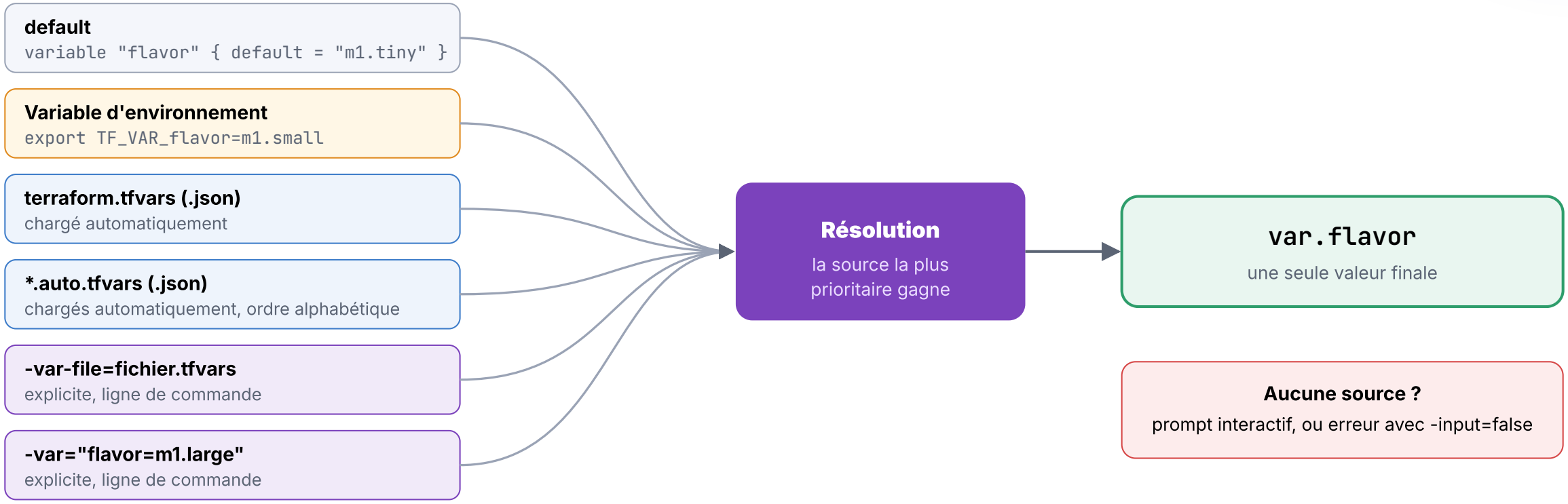


Les fichiers .tfvars et TF_VAR_ fournissent des valeurs, ils ne déclarent rien : chaque nom doit correspondre à un bloc `variable`. Un local, lui, ne peut pas être alimenté de l'extérieur.

Utopios® Tous droits réservés

Six sources pour une même variable

Une variable est déclarée une fois ; sa valeur peut venir de plusieurs endroits



Utopios® Tous droits réservés

terraform.tfvars et terraform.tfvars.json

terraform.tfvars — syntaxe HCL, uniquement des affectations

```
project        = "formation"
env            = "dev"
instance_count = 2
flavor         = "m1.small"

allowed_cidrs = ["10.0.0.0/8", "192.168.1.0/24"]

flavors = {
  dev  = "m1.small"
  prod = "m1.large"
}

vm = {
  name   = "bastion"
  flavor = "m1.small"
}
```

terraform.tfvars.json — même contenu en JSON

```
{
  "project": "formation",
  "env": "dev",
  "instance_count": 2,
  "allowed_cidrs": ["10.0.0.0/8"]
}
```

- Nom **exact** : `terraform.tfvars`. Un `dev.tfvars` n'est **pas** chargé automatiquement
- Lu uniquement dans le **répertoire courant** (ou celui donné par `-chdir`)
- Pas de bloc `variable`, pas de référence à `var.x` : uniquement des valeurs littérales
- La version `.json` est pratique quand les valeurs sont **générées par un script** ou un outil (CI, inventaire)
- Si les deux existent, le `.json` est lu **après** et l'emporte

Utopios® Tous droits réservés

*.auto.tfvars : chargement automatique multiple

répertoire de travail

```
.
├── variables.tf
├── terraform.tfvars          # 1
├── 10-network.auto.tfvars    # 2
├── 20-compute.auto.tfvars    # 3
├── 99-local.auto.tfvars      # 4  (ignoré par Git)
└── envs/
    └── prod.auto.tfvars      # jamais lu automatiquement
```

Ordre lexical des noms de fichiers : `10-…` avant `20-…` avant `99-…`. Si deux fichiers définissent la même variable, le dernier lu gagne.

- **Cas d'usage**
  - Découper de nombreuses valeurs par thème (réseau, compute, stockage)
  - Fichier généré par la CI (`ci.auto.tfvars`) déposé avant le `plan`
  - Surcharge personnelle non commitée (`99-local.auto.tfvars`)

Piège : un `*.auto.tfvars` oublié dans le dossier s'applique **silencieusement** à chaque plan. Lister les fichiers chargés en cas de valeur inattendue.

Préfixe numérique : rend l'ordre explicite au lieu de dépendre de l'alphabet (`compute` < `network`).

Utopios® Tous droits réservés

-var-file et -var : la ligne de commande

fichier par environnement

```
terraform plan  -var-file=envs/dev.tfvars
terraform apply -var-file=envs/prod.tfvars
```

valeur unique

```
terraform plan -var="flavor=m1.large"
terraform plan -var='allowed_cidrs=["10.0.0.0/8"]'
terraform plan -var='flavors={dev="m1.small",prod="m1.xl"}'
```

combinaison : traitées dans l'ordre d'apparition

```
terraform plan \
  -var-file=envs/common.tfvars \
  -var-file=envs/prod.tfvars \
  -var="instance_count=5"
# instance_count = 5 même si prod.tfvars le définit
```

- `-var` et `-var-file` ont la **même priorité** : c'est leur **position** sur la ligne de commande qui décide, le dernier gagne
- Plusieurs `-var-file` autorisés : commun puis spécifique
- Les types complexes passés en `-var` s'écrivent en **syntaxe HCL**
- Il faut passer **les mêmes** options à `plan`, `apply`, `destroy` et `console`

Guillemets : sous PowerShell, `-var='a=["x"]'` devient `-var='a=[\"x\"]'`. Préférer un fichier `-var-file` pour tout type complexe.

Plan sauvegardé : `terraform apply tfplan` refuse `-var` : les valeurs sont déjà figées dans le fichier de plan.

Utopios® Tous droits réservés

TF_VAR_ : les variables d'environnement

Linux / macOS

```
export TF_VAR_env=dev
export TF_VAR_instance_count=3
export TF_VAR_allowed_cidrs='["10.0.0.0/8","172.16.0.0/12"]'
export TF_VAR_vm='{ name = "bastion", flavor = "m1.small" }'
export TF_VAR_db_password="$(cat ~/.secrets/db)"
terraform plan
unset TF_VAR_db_password
```

Windows PowerShell

```
$env:TF_VAR_env = "dev"
$env:TF_VAR_allowed_cidrs = '["10.0.0.0/8"]'
Remove-Item Env:TF_VAR_env
```

- Format : `TF_VAR_` + nom **exact** de la variable (sensible à la casse sous Linux)
- Types complexes : valeur en **syntaxe HCL**, comme pour `-var`
- Priorité la **plus basse** après le default : n'importe quel tfvars l'écrase
- Une `TF_VAR_` qui ne correspond à aucune variable déclarée est **ignorée sans message**

Usage principal : les secrets en CI. Variable GitLab CI/CD masquée `TF_VAR_db_password` → jamais écrite dans un fichier ni dans l'historique du shell.

Ne pas confondre `TF_VAR_*` (valeurs de variables) avec les autres variables d'environnement Terraform : `TF_LOG`, `TF_CLI_ARGS`, `TF_INPUT`, `TF_DATA_DIR`, ni avec les `OS_*` lues par le provider.

Utopios® Tous droits réservés

L'ordre de priorité

Terraform lit les sources dans cet ordre ; chaque source lue écrase la précédente



Mémo

- Plus c'est explicite, plus c'est prioritaire
- CLI > fichiers auto > environnement > default
- TF_VAR_ est la source la plus faible
- Pas de fusion : une map ou un object est remplacé en entier
- Aucune valeur → prompt interactif → ou erreur en CI

Valable pour Terraform et OpenTofu

Utopios® Tous droits réservés

Exemple déroulé : qui gagne ?

variables.tf

```
variable "flavor" {
  type    = string
  default = "m1.tiny"
}
```

sources présentes

```
export TF_VAR_flavor=m1.small            # 1
# terraform.tfvars         : flavor = "m1.medium"   # 2
# 10-a.auto.tfvars         : flavor = "m1.large"    # 4
# 20-b.auto.tfvars         : flavor = "m1.xlarge"   # 4
# envs/prod.tfvars         : flavor = "m2.large"    # 5
```

COMMANDE	VAR.FLAVOR
`terraform plan`	**m1.xlarge** (20-b lu après 10-a)
`plan -var-file=envs/prod.tfvars`	**m2.large**
`plan -var-file=envs/prod.tfvars -var=flavor=m2.xl`	**m2.xl**
`plan -var=flavor=m2.xl -var-file=envs/prod.tfvars`	**m2.large** (ordre inversé)
sans les `*.auto.tfvars` ni CLI	**m1.medium**
sans aucun fichier ni CLI	**m1.small** (TF_VAR_)
`unset TF_VAR_flavor` en plus	**m1.tiny** (default)

- **Vérifier la valeur effective**

```
terraform console -var-file=envs/prod.tfvars
> var.flavor
"m2.large"
```

- **Lister les fichiers auto-chargés**

```
ls terraform.tfvars* *.auto.tfvars* 2>/dev/null
env | grep '^TF_VAR_'
```

Astuce de formation : faire prédire le résultat de chaque ligne avant de lancer `terraform console` .

Utopios® Tous droits réservés

Pièges fréquents

1 · Pas de fusion des maps et objects

```
# terraform.tfvars
tags = { owner = "ops", env = "dev" }
# prod.tfvars
tags = { env = "prod" }
# résultat : { env = "prod" } → owner perdu
```

Solution : fusionner dans un local avec `merge(var.base_tags, var.env_tags)`.

2 · Les tfvars ne concernent que le root module

Un module enfant ne lit **aucun** fichier tfvars ni `TF_VAR_` : ses variables sont alimentées uniquement par les arguments du bloc `module`.

```
module "web" {
  source = "./modules/vm"
  flavor = var.flavor   # transmission explicite
}
```

3 · Variable non déclarée

dans un `.tfvars`	warning
avec `-var`	erreur
avec `TF_VAR_`	ignorée sans message

Une faute de frappe dans `TF_VAR_flavour` passe donc inaperçue.

4 · Prompt interactif en CI

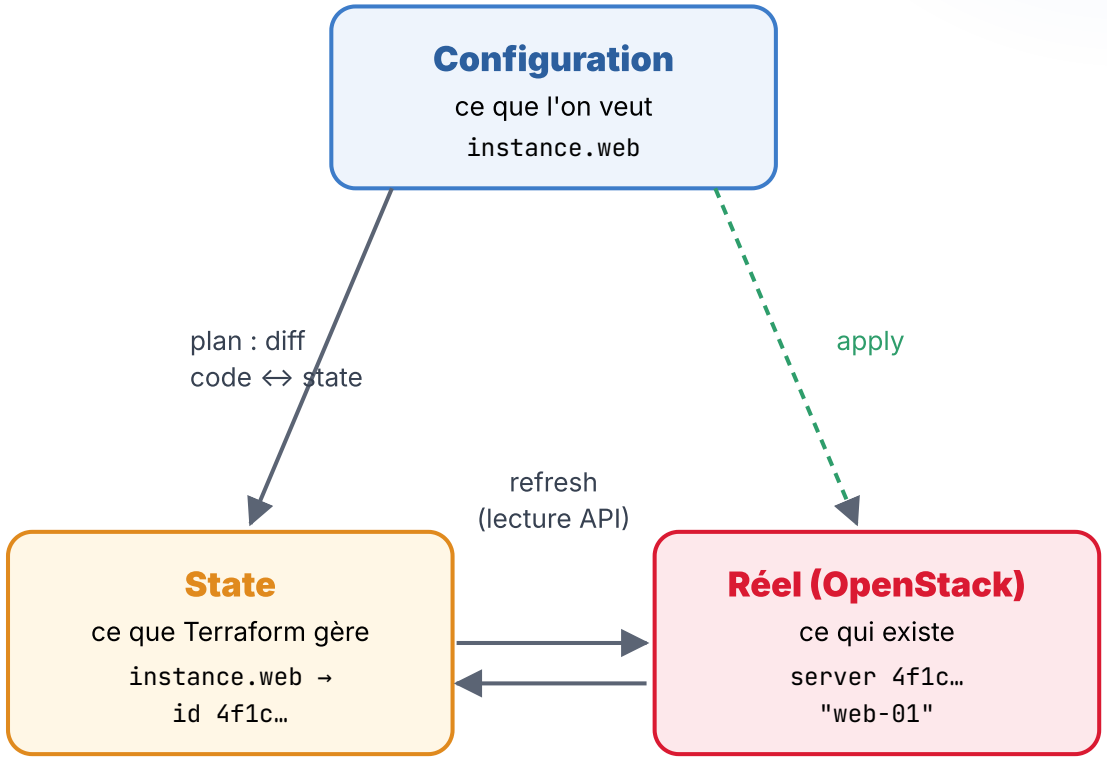
Une variable obligatoire sans valeur bloque le job en attente de saisie. En CI : `-input=false` ou `TF_INPUT=0` pour échouer immédiatement.

5 · Secrets commités : un `terraform.tfvars` avec un mot de passe finit dans Git. Secrets = `TF_VAR_` masquées en CI, ou fichier ignoré par Git.

Utopios® Tous droits réservés

Comprendre et gérer l'état de Terraform

- L'état de Terraform est un aspect crucial de Terraform.
- Il garde une trace de l'infrastructure que Terraform a provisionnée et est utilisé pour planifier et appliquer des modifications à cette infrastructure.
- L'état de Terraform est un fichier JSON qui contient les propriétés actuelles de l'infrastructure provisionnée. Il est utilisé par Terraform pour mapper les ressources réelles aux ressources dans votre configuration, pour stocker les attributs des ressources managées, et pour optimiser les performances pour les grandes infrastructures.
- Par défaut, Terraform stocke l'état localement dans un fichier appelé `terraform.tfstate`. Cependant, il est recommandé de stocker l'état dans un emplacement distant, tel qu'un bucket S3, Azure Blob Storage ou le state géré par GitLab, pour des raisons de sécurité, de performance et de collaboration.

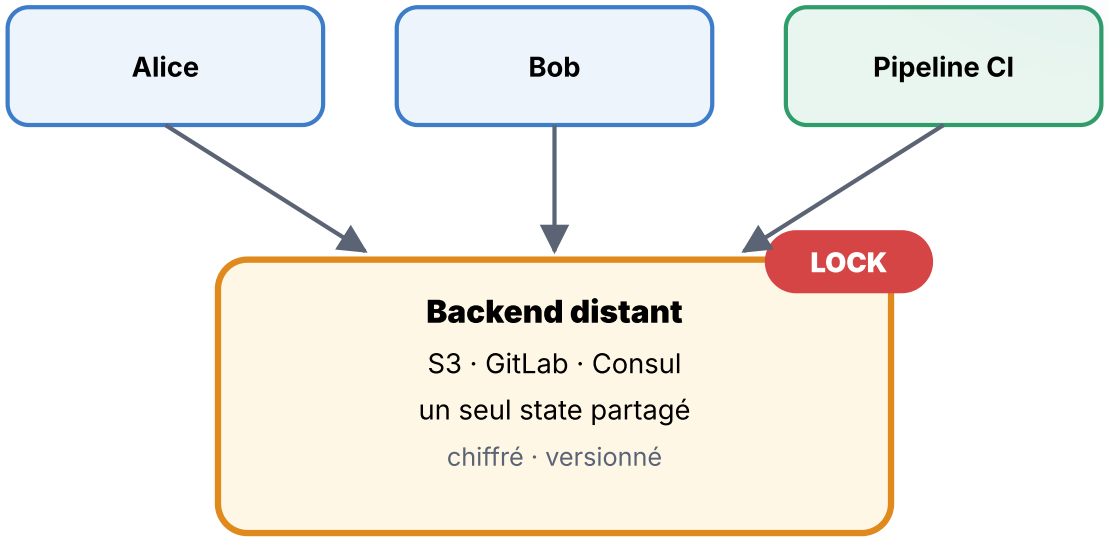


Utopios® Tous droits réservés

Comprendre et gérer l'état de Terraform

Gestion de l'état de Terraform

- **Stockage distant :** Vous pouvez configurer Terraform pour stocker l'état dans un emplacement distant. Cela est particulièrement important lorsque vous travaillez en équipe, car il permet de partager l'état de Terraform entre les membres de l'équipe.
- **Verrouillage d'état :** Le verrouillage d'état empêche d'autres utilisateurs de modifier l'état de Terraform en même temps. Certains backends, comme S3 (lock natif `use_lockfile`, anciennement avec DynamoDB) ou GitLab, prennent en charge le verrouillage d'état.
- **Séparation de l'état :** Pour les grandes infrastructures, il peut être utile de séparer l'état de Terraform en plusieurs fichiers d'état, avec les workspaces de Terraform ou des configurations distinctes pour différentes parties de votre infrastructure.
- Vous pouvez inspecter l'état actuel avec la commande `terraform show`.
  - `terraform state list` : Liste toutes les ressources dans l'état de Terraform.
  - `terraform state show` : Affiche les détails d'une ressource spécifique.
  - `terraform state mv` : Déplace une ressource vers un autre emplacement dans l'état.
  - `terraform state rm` : Supprime une ressource de l'état de Terraform.



un seul plan / apply à la fois

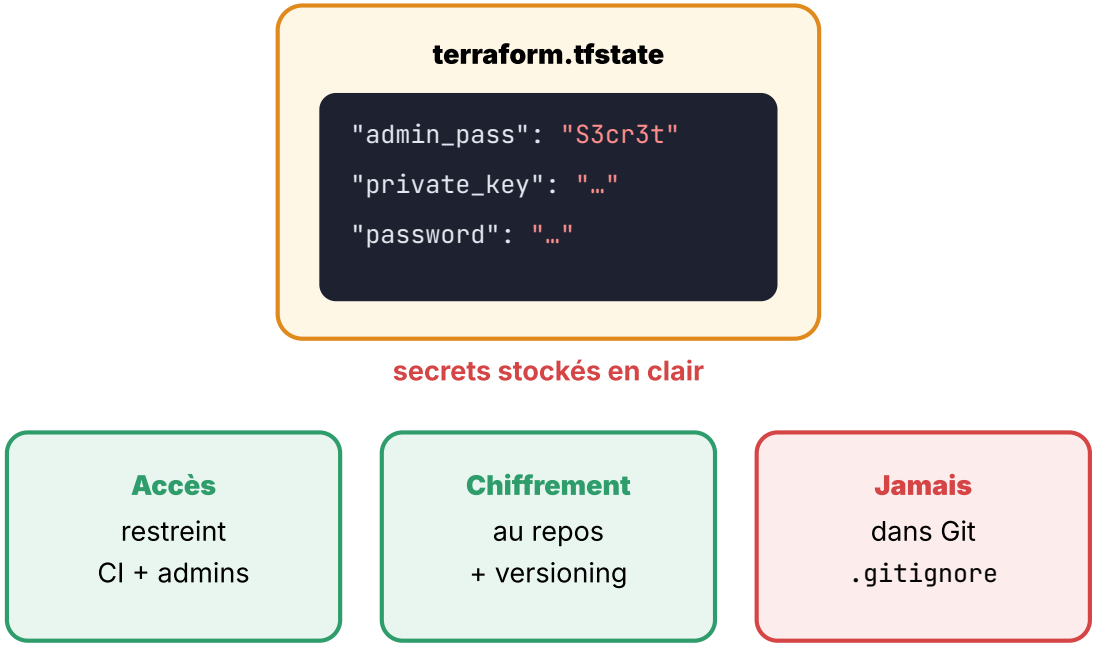
```
terraform state list # lister
terraform state show ADDR # détails
terraform state mv SRC DST # renommer
terraform state rm ADDR # oublier
```

Utopios® Tous droits réservés

Comprendre et gérer l'état de Terraform

Sécurisation de l'état de Terraform

- L'état de Terraform peut contenir des informations sensibles, comme des mots de passe ou des clés d'accès. Il est donc important de sécuriser l'état de Terraform.
- Stockez l'état dans un emplacement sécurisé et contrôlez l'accès à cet emplacement.
- Utilisez le chiffrement pour stocker l'état.
- Ne stockez jamais l'état de Terraform dans un dépôt de code public ou non sécurisé.



Utopios® Tous droits réservés

Comprendre et gérer l'état de Terraform EXERCICE

Exercice

- Créez un fichier `main.tf` avec une ressource fictive :

```
resource "null_resource" "example" {
  triggers = {
    always_run = "${timestamp()}"
  }
}
```

- Listez toutes les ressources dans votre état de Terraform.
- Affichez les détails de la ressource `null_resource.example`.
- Supprimez la ressource `null_resource`.

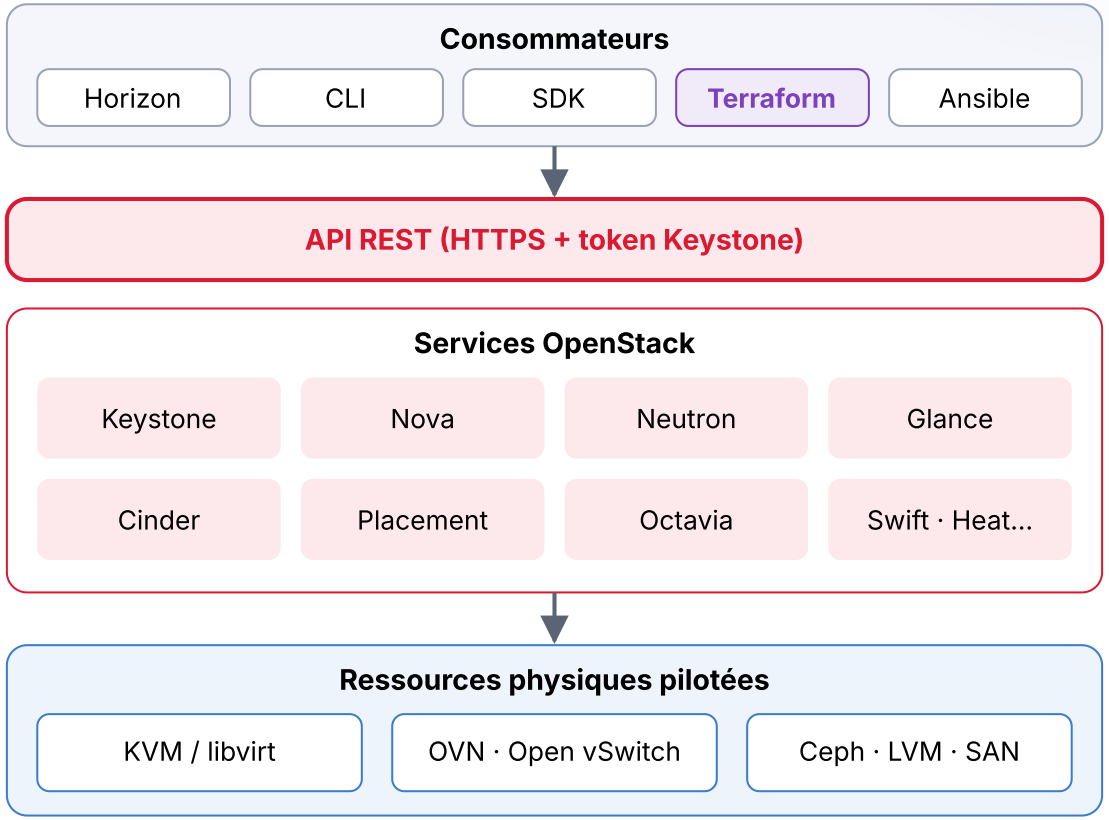
Utopios® Tous droits réservés

Qu'est-ce qu'OpenStack ?

Une plateforme open source pour construire un cloud IaaS privé ou public

- **IaaS** : fournit à la demande des machines virtuelles, des réseaux, du stockage bloc et objet, des load balancers
- **Open source** (Apache 2.0), créé en 2010 par la NASA et Rackspace, gouverné par l'**OpenInfra Foundation** (Linux Foundation)
- **Modulaire** : un ensemble de services indépendants (Nova, Neutron, Cinder…), chacun avec son API REST
- **API-first** : tout ce que fait Horizon passe par les API, donc tout est automatisable (CLI, SDK, Terraform)
- **Multi-tenant** : isolation des projets, quotas, rôles
- **Deux versions par an**, nommées `AAAA.1` / `AAAA.2` : 2025.1 Epoxy, 2025.2 Flamingo, 2026.1 Gazpacho, 2026.2 Hibiscus. Une version sur deux est **SLURP** (montée de version en sautant une release)

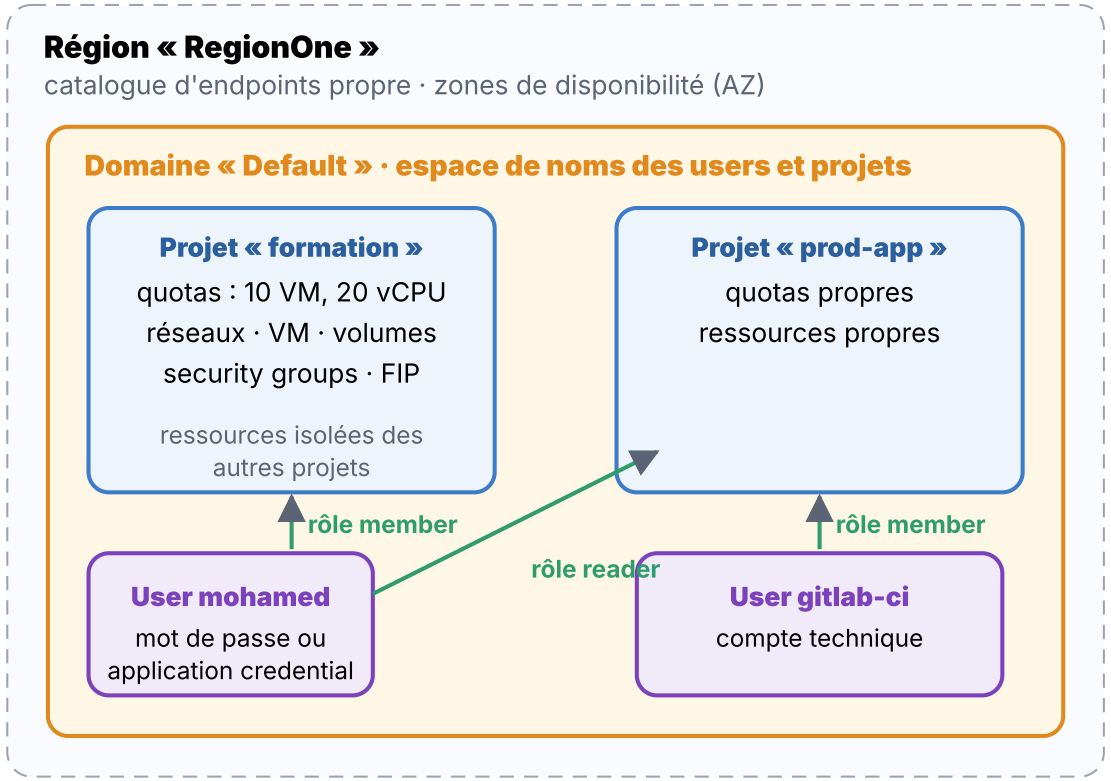
Utilisateurs : opérateurs télécoms, clouds publics européens (OVHcloud, Infomaniak, Cleura…), recherche (CERN), industrie et défense pour des clouds souverains ou isolés.



Utopios® Tous droits réservés

Les concepts d'identité : domaine, projet, user, rôle

Gérés par Keystone ; ils déterminent où Terraform crée les ressources et ce qu'il a le droit de faire



CONCEPT	RÔLE
Domaine	Conteneur de users, groupes et projets (souvent un par organisation)
Projet (ex-tenant)	Unité d'isolation et de quotas : toutes les ressources y sont rattachées
User / groupe	Identité qui s'authentifie
Rôle	Droits d'un user **sur un projet** : `reader`, `member`, `manager`, `admin`
Token	Jeton temporaire, scopé sur un projet, envoyé à chaque appel API
Région	Ensemble de services avec ses propres endpoints
AZ	Groupe d'hyperviseurs ou de stockage dans une région (domaine de panne)
Quota	Limite par projet : instances, vCPU, RAM, volumes, FIP, LB…

Pour Terraform : les credentials du provider désignent un user **et** un projet. Avec le rôle `reader`, le `plan` fonctionne mais l'`apply` échoue en 403.

Utopios® Tous droits réservés

Les services principaux

SERVICE	TYPE	RÔLE	OBJETS MANIPULÉS	PRÉFIXE TERRAFORM
Keystone	identity	Authentification, tokens, catalogue des endpoints, projets et rôles	domain, project, user, role	`identity_*_v3`
Nova	compute	Cycle de vie des machines virtuelles, ordonnancement sur les hyperviseurs	server, flavor, keypair, server group	`compute_*_v2`
Placement	placement	Inventaire des ressources disponibles (vCPU, RAM, disque) pour le scheduler	resource provider	— (interne)
Neutron	network	Réseaux virtuels, subnets, routers, ports, security groups, floating IP	network, subnet, router, port, FIP	`networking_*_v2`
Glance	image	Catalogue des images disque de démarrage	image	`images_*_v2`
Cinder	volumev3	Stockage bloc persistant, snapshots, types de volume	volume, snapshot, volume type	`blockstorage_*_v3`
Octavia	load-balancer	Load balancing L4 / L7 (amphora ou OVN)	LB, listener, pool, member	`lb_*_v2`
Swift	object-store	Stockage objet (API Swift et S3)	container, object	`objectstorage_*_v1`
Horizon	dashboard	Interface web, cliente des API	—	—
Heat	orchestration	IaC native OpenStack (stacks HOT)	stack	— (concurrent)
Autres		Designate (DNS), Barbican (secrets), Manila (fichiers), Magnum (Kubernetes), Ironic (bare metal)		`dns_*` , `keymanager_*` …

Le **type** est le nom du service dans le catalogue Keystone (`openstack catalog list`) : c'est lui que le provider cherche pour trouver l'URL de chaque API. Un service absent du catalogue = ressources Terraform correspondantes inutilisables.

Utopios® Tous droits réservés

Architecture d'un déploiement

Des nœuds spécialisés reliés par des réseaux dédiés

Nœuds de contrôle (×3 en HA)

API des services

keystone · nova-api · neutron-server
glance-api · cinder-api · octavia-api
placement · horizon

Agents de contrôle

nova-scheduler · nova-conductor
cinder-scheduler · octavia-worker
OVN northd

Infrastructure partagée

MariaDB / Galera (état des services) · RabbitMQ (bus de messages RPC)
Memcached (cache des tokens) · HAProxy + VIP (point d'entrée des API)

Nœuds compute (×N)

nova-compute → libvirt → QEMU/KVM : exécute les VM
ovn-controller / Open vSwitch : réseau des VM
metadata agent (cloud-init, user_data)

Nœuds de stockage

cinder-volume → backend Ceph RBD, LVM/iSCSI, baie SAN
Ceph : images Glance, volumes Cinder, disques éphémères Nova
Swift ou Ceph RGW : stockage objet (S3)

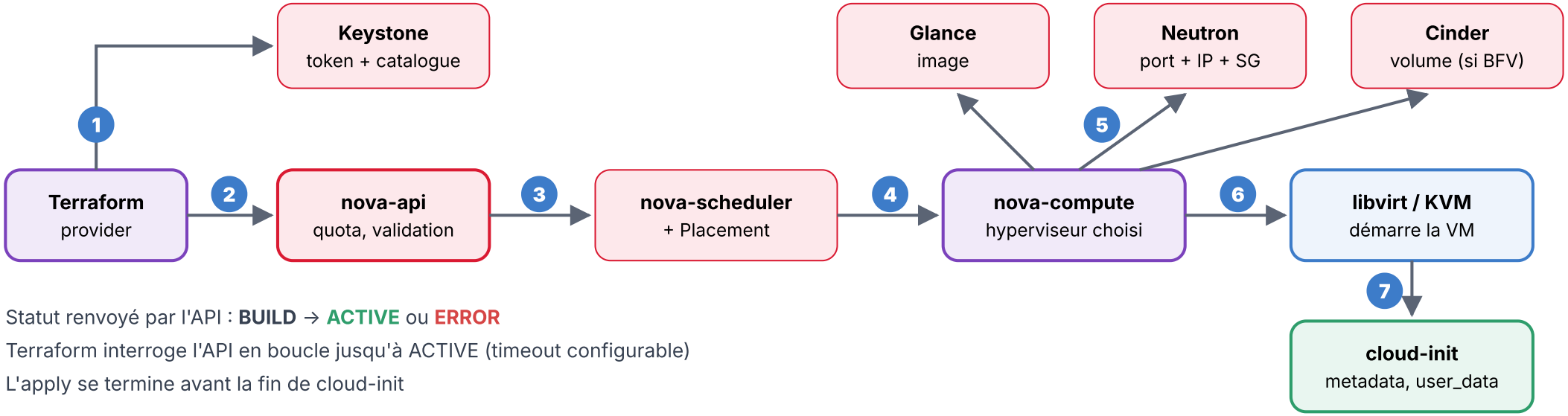
Management / API	communication entre services, base, RabbitMQ, appels API internes
Tunnel / overlay	trafic entre VM de réseaux self-service (Geneve / VXLAN)
Storage	trafic Ceph / iSCSI entre compute et stockage
External / provider	réseau « public » : floating IP, passerelle des routers, endpoints API publics (ce que Terraform appelle)

Ce que voit Terraform : uniquement les **endpoints API publics** derrière HAProxy. Toute la mécanique interne (RabbitMQ, scheduler, agents) explique les délais de création et les états `BUILD`, `PENDING_CREATE`, `ERROR`.

Utopios® Tous droits réservés

Ce qui se passe quand on crée une instance

Le même enchaînement pour `openstack server create`, Horizon ou `openstack_compute_instance_v2`



Statut renvoyé par l'API : **BUILD** → **ACTIVE** ou **ERROR**

Terraform interroge l'API en boucle jusqu'à ACTIVE (timeout configurable)

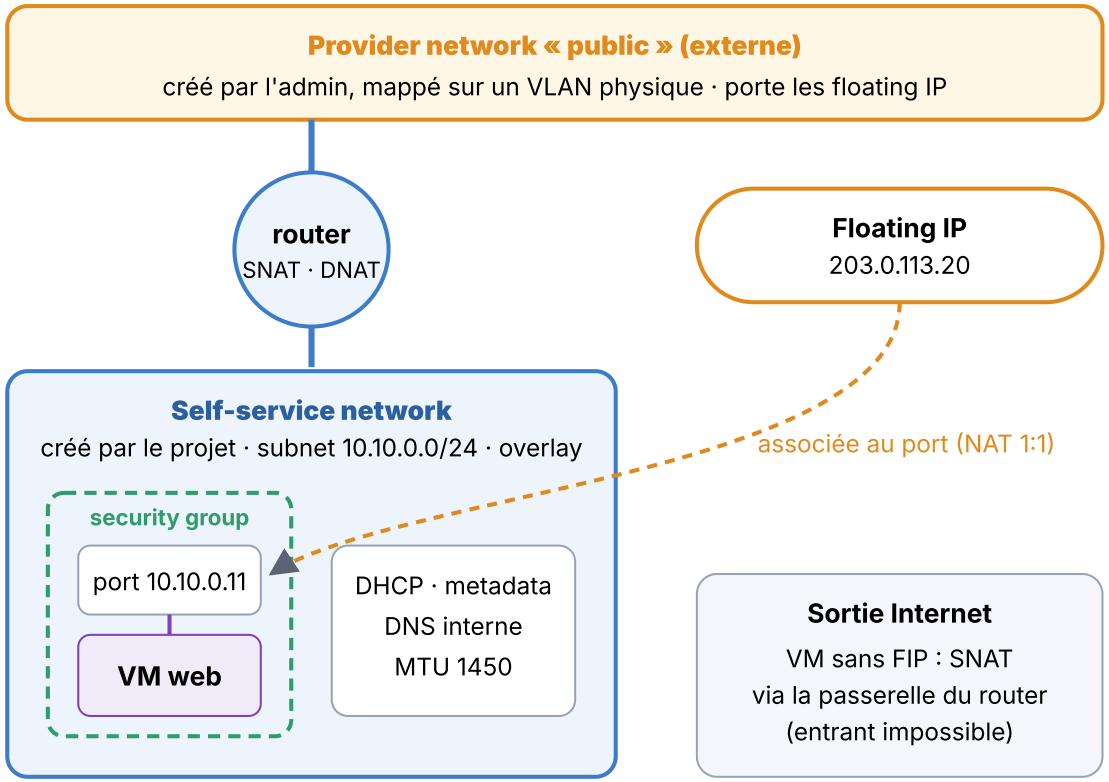
L'apply se termine avant la fin de cloud-init

1 Authentification, récupération du token et des endpoints **2** `POST /servers`, contrôle des quotas

3–4 Choix de l'hyperviseur via Placement, message RabbitMQ **5** Image, port Neutron, volume **6–7** Démarrage KVM puis cloud-init

Utopios® Tous droits réservés

Le réseau Neutron en bref



OBJET	RÔLE
Network	Segment de niveau 2 isolé
Subnet	Plage IP, passerelle, DHCP, DNS d'un network
Port	Point de connexion : MAC + IP + security groups (une NIC de VM, une interface de router, un VIP)
Router	Relie des subnets et le réseau externe (SNAT, floating IP)
Floating IP	IP publique associée à un port (NAT 1:1)
Security group	Pare-feu à état appliqué sur chaque port ; par défaut tout l'entrant est bloqué

Provider network (admin, réseau physique) vs **self-service network** (projet, overlay) : Terraform crée le second et ne fait que **lire** le premier avec une data source.

Ordre obligatoire : network → subnet → router + interface → port → VM → FIP. C'est exactement le graphe de dépendances que Terraform devra reconstruire.

Utopios® Tous droits réservés

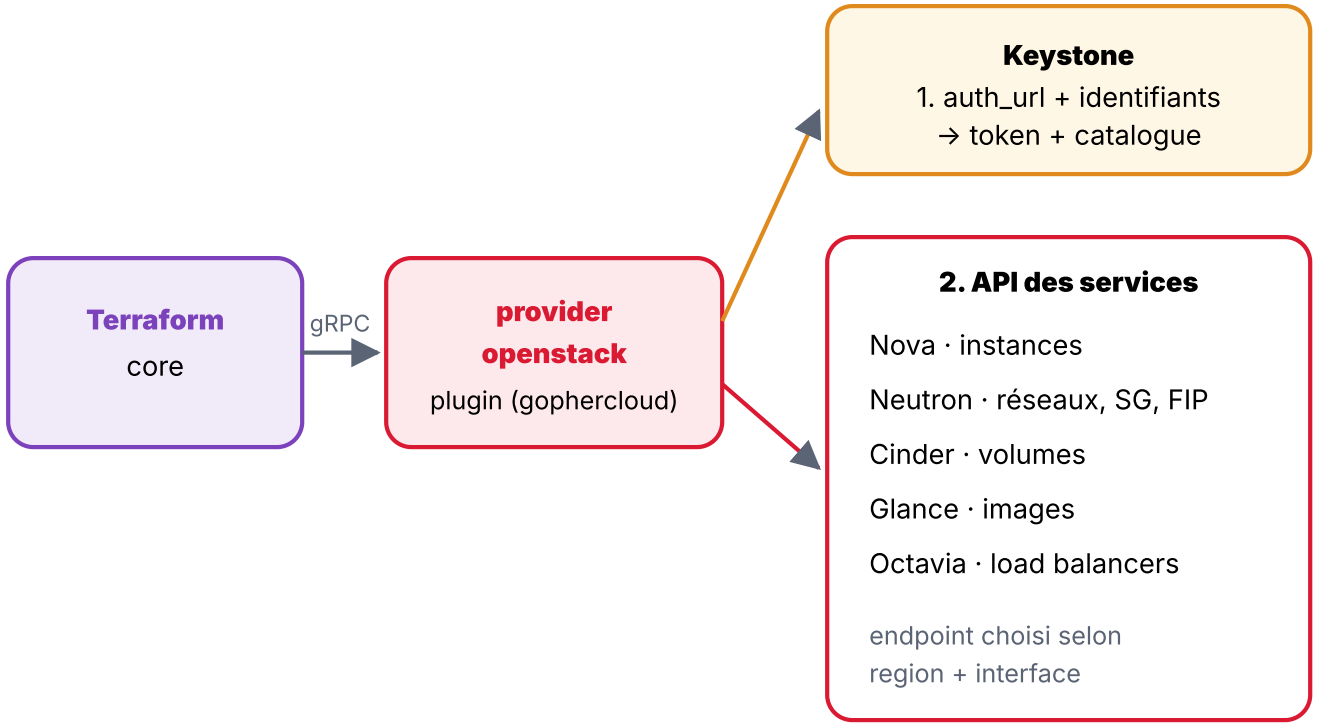
Gestion des ressources openstack avec Terraform - Provider

- Un provider dans Terraform agit comme un intermédiaire entre Terraform et une plateforme spécifique. Pour OpenStack, le provider est `openstack`. Ce provider définit les ressources spécifiques à OpenStack.
- Pour interagir avec les ressources d'OpenStack, vous utiliserez le provider openstack de Terraform.

```hcl
terraform {
  required_providers {
    openstack = {
      source  = "terraform-provider-openstack/openstack"
      version = "~> 3.0"
    }
  }
}

provider "openstack" {
  user_name   = ""
  tenant_name = ""
  password    = ""
  auth_url    = ""
  region      = "microstack"
}
```

> Mot de passe : préférer les variables `OS_*` (fichier `openrc`) ou `clouds.yaml` plutôt que la valeur en clair.



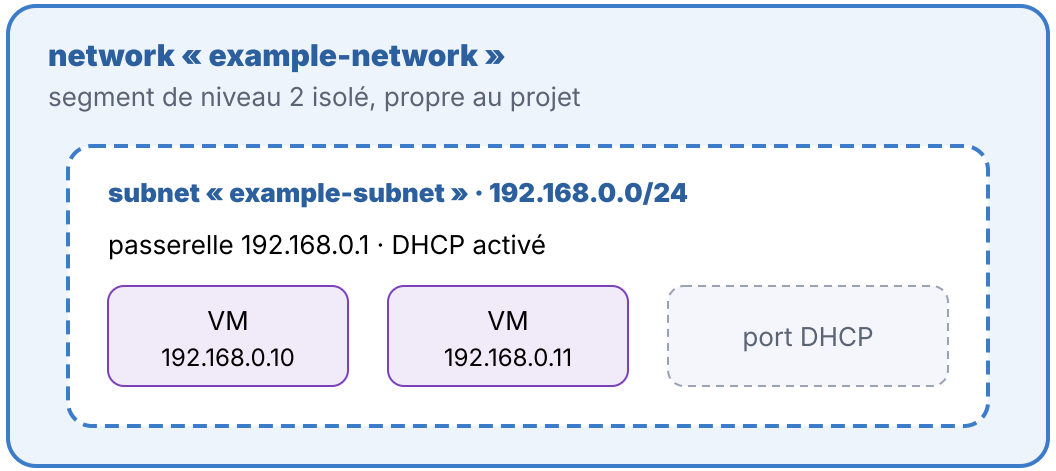
Utopios® Tous droits réservés

Création d'un réseau et d'un sous-réseau

- Créer un réseau avec `openstack_networking_network_v2`
- Créer un sous-réseau avec `openstack_networking_subnet_v2`
- Exemple de configuration :

```
resource "openstack_networking_network_v2" "example_network" {
  name = "example-network"
}

resource "openstack_networking_subnet_v2" "example_subnet" {
  name       = "example-subnet"
  network_id = openstack_networking_network_v2.example_network.id
  cidr       = "192.168.0.0/24"
}
```



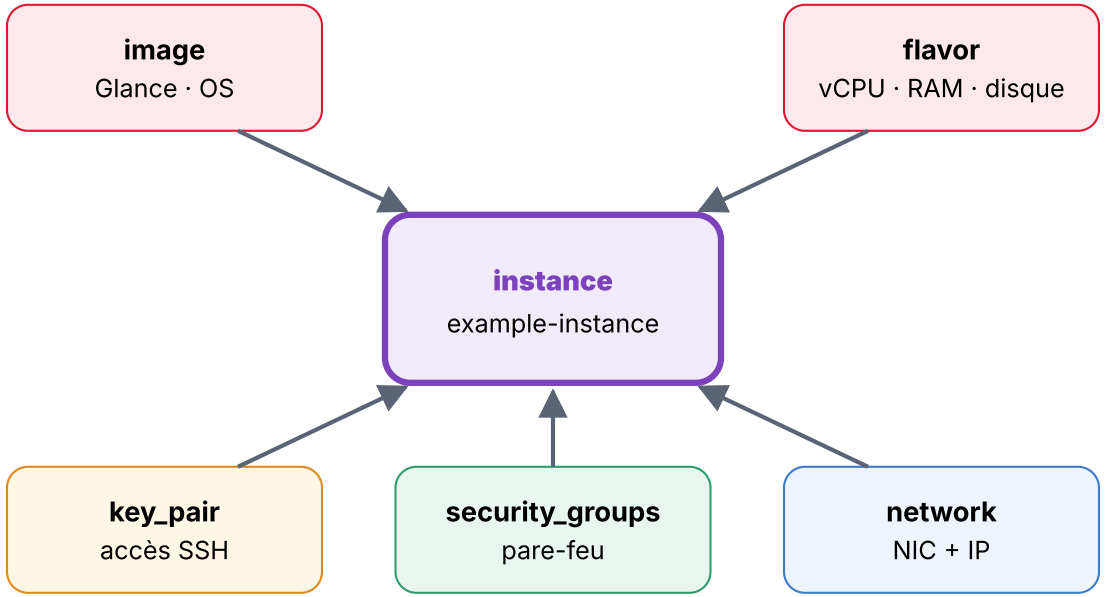
Utopios® Tous droits réservés

Création d'une instance

- Créer une instance avec `openstack_compute_instance_v2`
- Spécifier l'image, le flavor, le réseau et le groupe de sécurité
- Exemple de configuration :

```
resource "openstack_compute_instance_v2" "example" {
  name            = "example-instance"
  image_id        = "image_id"
  flavor_id       = "flavor_id"
  key_pair        = "key_pair_name"
  security_groups = ["default", "example_security_group"]

  network {
    uuid = openstack_networking_network_v2.example_network.id
  }
}
```



Utopios® Tous droits réservés

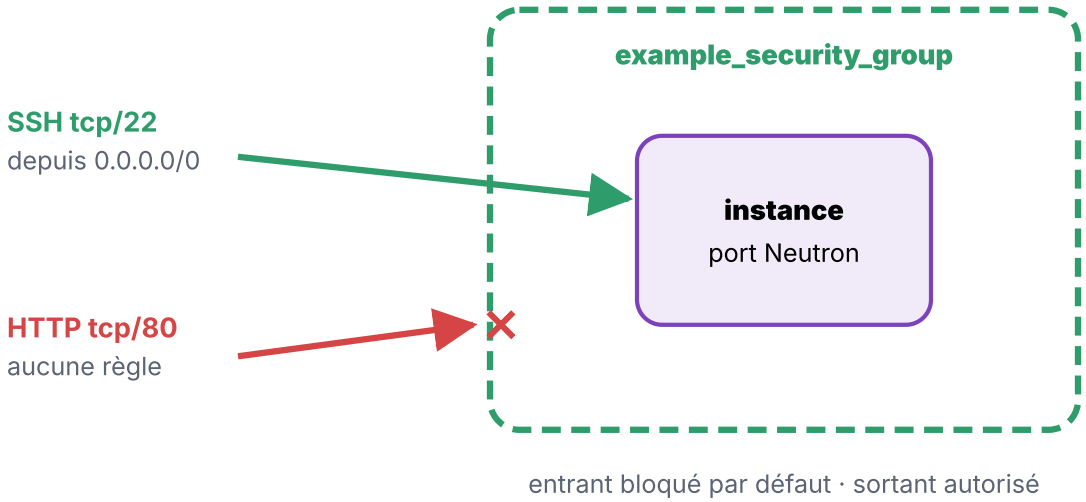
Création d'un groupe de sécurité

- Créer un groupe de sécurité avec `openstack_networking_secgroup_v2`
- Ajouter des règles avec `openstack_networking_secgroup_rule_v2`
- Exemple de configuration :

```
resource "openstack_networking_secgroup_v2" "example" {
  name        = "example_security_group"
  description = "Example security group"
}

resource "openstack_networking_secgroup_rule_v2" "example_rule" {
  direction         = "ingress"
  ethertype         = "IPv4"
  protocol          = "tcp"
  port_range_min    = 22
  port_range_max    = 22
  remote_ip_prefix  = "0.0.0.0/0"
  security_group_id = openstack_networking_secgroup_v2.example.id
}
```

Mise à jour : `openstack_compute_secgroup_v2` et `openstack_compute_secgroup_rule_v2` ont été supprimés du provider 3.x ; on utilise les ressources Neutron.



Utopios® Tous droits réservés

Exercice 2 - Création d'une instance simple

- Créer un fichier de configuration Terraform pour déployer une instance OpenStack
- Utiliser le provider OpenStack et la ressource `openstack_compute_instance_v2`
- Appliquer la configuration

Utopios® Tous droits réservés

Exercice 3 - Configuration d'un réseau et d'un sous-réseau

- Modifier la configuration de l'exercice 2 Terraform pour inclure un réseau et un sous-réseau personnalisés
- Utiliser les ressources `openstack_networking_network_v2` et `openstack_networking_subnet_v2`
- Appliquer les modifications

Utopios® Tous droits réservés

Exercice 4 - Ajout d'un groupe de sécurité

- Ajouter un groupe de sécurité à la configuration Terraform
- Utiliser les ressources `openstack_networking_secgroup_v2` et `openstack_networking_secgroup_rule_v2`
- Appliquer les modifications

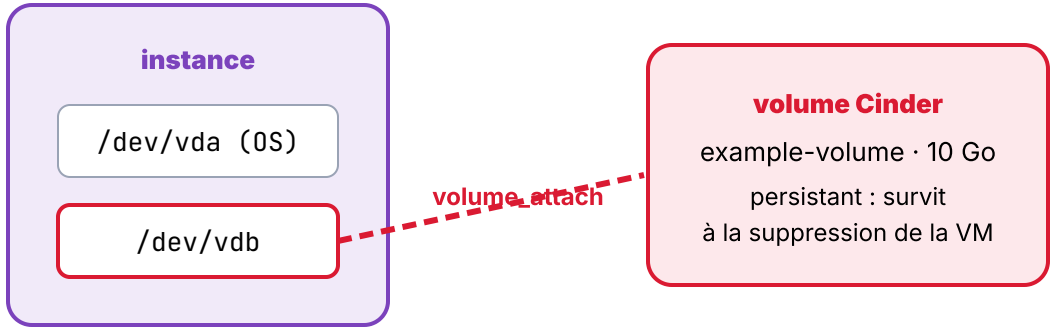
Utopios® Tous droits réservés

Création d'un volume

- Créer un volume avec `openstack_blockstorage_volume_v3`
- Attacher le volume à l'instance avec `openstack_compute_volume_attach_v2`
- Exemple de configuration :

```
resource "openstack_blockstorage_volume_v3" "example" {
  name = "example-volume"
  size = 10
}

resource "openstack_compute_volume_attach_v2" "example_attach" {
  instance_id = openstack_compute_instance_v2.example.id
  volume_id   = openstack_blockstorage_volume_v3.example.id
}
```



Utopios® Tous droits réservés

Exercice 6 - Création de volumes

- Créez un fichier pour définir les variables nécessaires.
- Créez un fichier pour définir les ressources suivantes :
  - `openstack_blockstorage_volume_v3` (volume)
  - `openstack_compute_instance_v2` (instance)
  - `openstack_compute_volume_attach_v2` (attachement du volume)
- Utilisez les variables pour paramétrer les ressources.
- Configurez la taille du volume, le type de volume et les autres paramètres nécessaires.
- Déployez l'infrastructure.
- Vérifiez que le volume a été créé et attaché à l'instance dans OpenStack.
- Supprimez l'infrastructure.

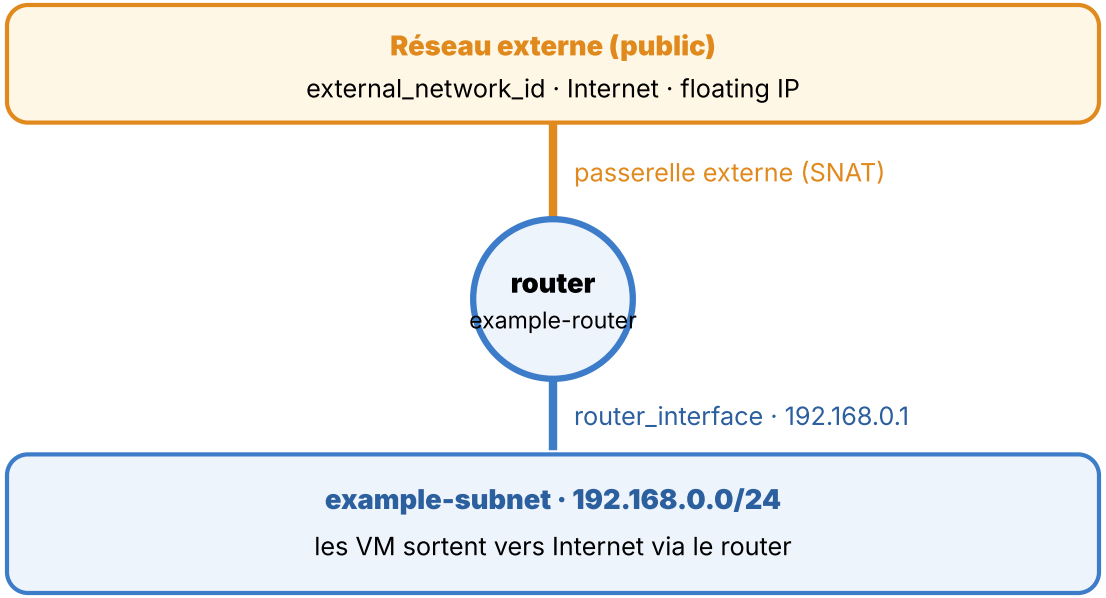
Utopios® Tous droits réservés

Création d'un routeur et connexion au réseau externe

- Créer un routeur avec `openstack_networking_router_v2`
- Connecter le routeur au sous-réseau avec `openstack_networking_router_interface_v2`
- Exemple de configuration :

```
resource "openstack_networking_router_v2" "example" {
  name                = "example-router"
  external_network_id = "external_network_id"
}

resource "openstack_networking_router_interface_v2" "example_interface" {
  router_id = openstack_networking_router_v2.example.id
  subnet_id = openstack_networking_subnet_v2.example_subnet.id
}
```

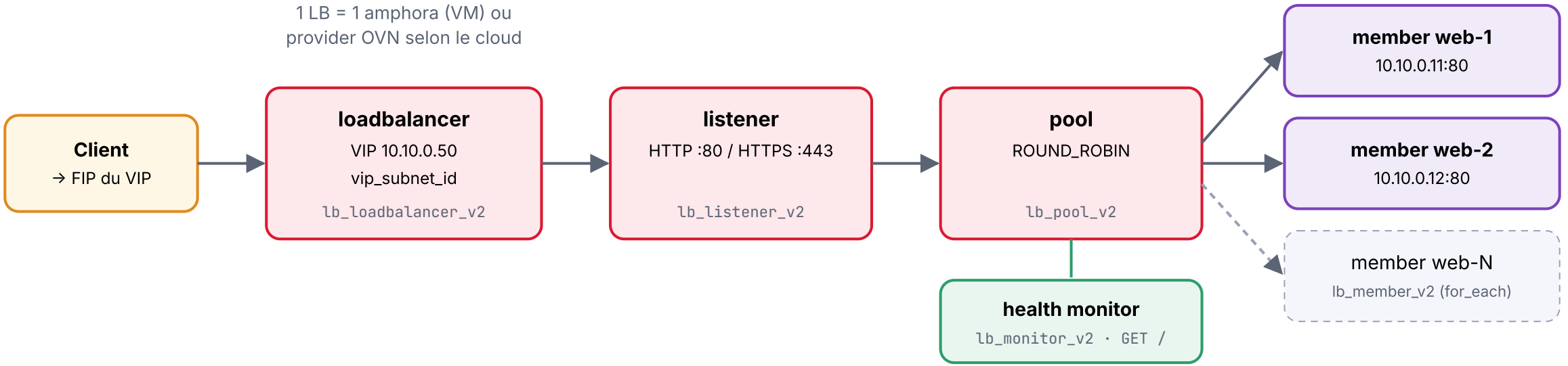


Corrigé : l'attribut s'appelle `external_network_id`, et l'interface a besoin de `router_id = …router_v2.example.id` et d'un `subnet_id`.

Utopios® Tous droits réservés

Création d'un équilibreur de charge

- Créer un équilibreur de charge avec `openstack_lb_loadbalancer_v2`
- Ajouter des auditeurs avec `openstack_lb_listener_v2`
- Ajouter des pools avec `openstack_lb_pool_v2`
- Ajouter des membres au pool avec `openstack_lb_member_v2`
- Exemple de configuration :



Utopios® Tous droits réservés

Création d'un équilibreur de charge

```
resource "openstack_lb_loadbalancer_v2" "example" {
  name          = "example-loadbalancer"
  vip_subnet_id = openstack_networking_subnet_v2.example_subnet.id
}

resource "openstack_lb_listener_v2" "example" {
  name            = "example-listener"
  protocol        = "HTTP"
  protocol_port   = 80
  loadbalancer_id = openstack_lb_loadbalancer_v2.example.id
}
```

```
resource "openstack_lb_pool_v2" "example" {
  name        = "example-pool"
  protocol    = "HTTP"
  lb_method   = "ROUND_ROBIN"
  listener_id = openstack_lb_listener_v2.example.id
}

resource "openstack_lb_member_v2" "example" {
  address       = "192.168.0.10"
  protocol_port = 80
  subnet_id     = openstack_networking_subnet_v2.example_subnet.id
  pool_id       = openstack_lb_pool_v2.example.id
}
```

Utopios® Tous droits réservés

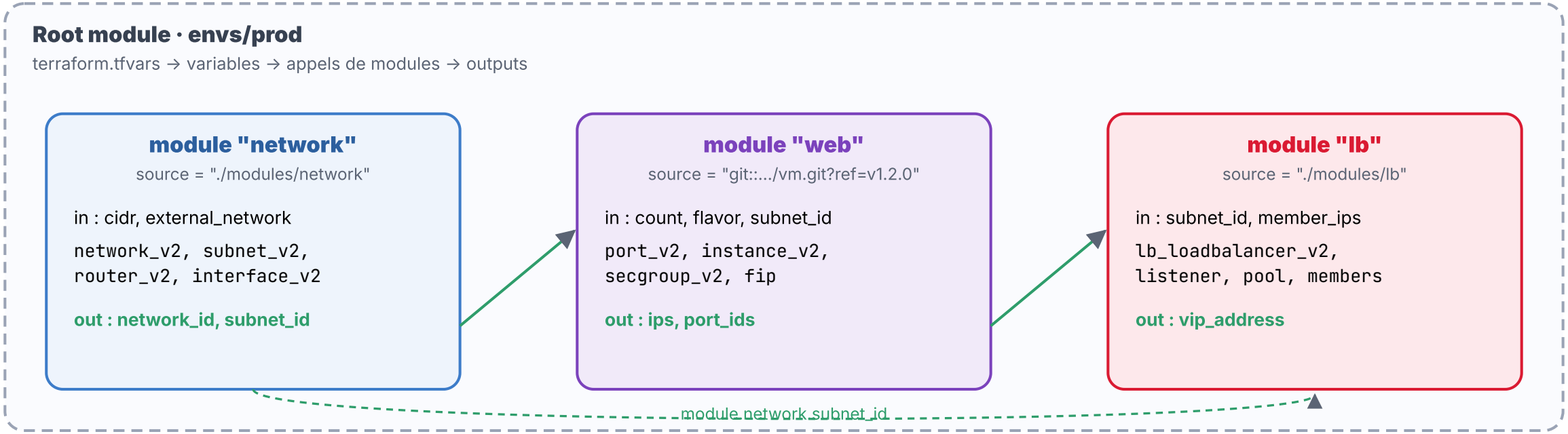
Exercice 7 - Création d'un équilibreur de charge

- Créez un fichier pour définir les variables nécessaires.
- Créez un fichier pour définir les ressources suivantes :
  - `openstack_networking_network_v2` (réseau)
  - `openstack_networking_subnet_v2` (sous-réseau)
  - `openstack_compute_instance_v2` (2 instances)
  - `openstack_lb_loadbalancer_v2` (équilibreur de charge)
  - `openstack_lb_listener_v2` (auditeur)
  - `openstack_lb_pool_v2` (pool)
  - `openstack_lb_member_v2` (membres du pool)
- Utilisez les variables pour paramétrer les ressources.
- Déployez l'infrastructure.
- Supprimez l'infrastructure.

Utopios® Tous droits réservés

Bonnes pratiques - Utilisation de modules

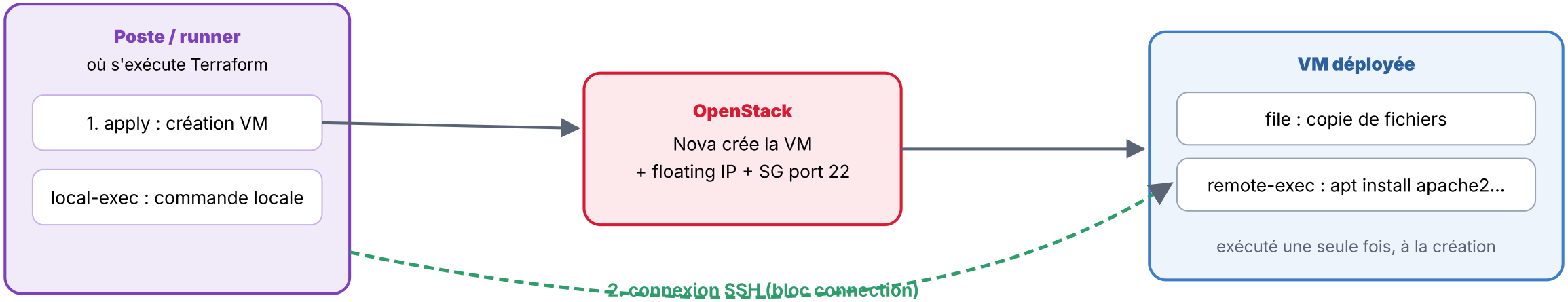
- Utiliser des modules pour organiser et réutiliser des configurations Terraform
- Créer des modules personnalisés ou utiliser des modules existants dans le registre Terraform



Utopios® Tous droits réservés

Terraform Provisioner pour Openstack

- **Description :** Les provisionneurs Terraform dans OpenStack sont utilisés pour effectuer des tâches post-déploiement sur des machines virtuelles. Cela est souvent nécessaire pour l'initialisation de systèmes ou l'exécution de scripts de configuration.
- **Exemple d'utilisation :** Pour provisionner une machine virtuelle OpenStack, on peut utiliser le provisionneur `remote-exec` pour exécuter des commandes ou des scripts sur la VM une fois qu'elle est déployée.



Utopios® Tous droits réservés

Options de Provisionnement dans openstack

- **Stratégies de provisionnement :** Dans OpenStack, plusieurs stratégies peuvent être utilisées pour le provisionnement des VMs, comme le démarrage à partir de templates, la personnalisation du système d'exploitation, ou l'exécution de scripts post-déploiement.
- **Personnalisation :** Terraform permet de personnaliser les VMs OpenStack lors de leur création, notamment en termes de configuration réseau, d'options de sécurité, et de paramètres de système d'exploitation.



Utopios® Tous droits réservés

Terraform Provisioner

- `local-exec` : Ce provisionneur exécute des commandes sur la machine locale, c'est-à-dire la machine qui exécute Terraform.
- `remote-exec` : Ce provisionneur exécute des commandes sur une machine distante.
- `file` : Ce provisionneur permet de copier des fichiers ou des répertoires d'une machine locale vers une machine distante.

> L'utilisation des provisionneurs est souvent déconseillée, car elle peut entraîner des configurations difficilement reproductibles. Il est généralement préférable d'utiliser des outils de configuration tels que Ansible, Chef ou Puppet pour configurer les machines virtuelles après leur création.

```
resource "openstack_compute_instance_v2" "web" {
  # ... image, flavor, key_pair, network

  connection {
    type        = "ssh"
    host        = self.access_ip_v4
    user        = "ubuntu"
    private_key = file("~/.ssh/id_rsa")
  }

  provisioner "file" {
    source      = "index.html"
    destination = "/tmp/index.html"
  }

  provisioner "remote-exec" {
    inline = [
      "sudo apt-get update -y",
      "sudo apt-get install -y apache2",
      "sudo mv /tmp/index.html /var/www/html/",
    ]
  }

  provisioner "local-exec" {
    command = "echo ${self.access_ip_v4} >> hosts.txt"
  }
}
```

Utopios® Tous droits réservés

Terraform Provisioner - Scénario Pratique EXERCICE

Contexte : Vous êtes un ingénieur DevOps travaillant sur un projet d'infrastructure OpenStack. Votre tâche consiste à automatiser la configuration des VMs après leur déploiement.

Exercice :

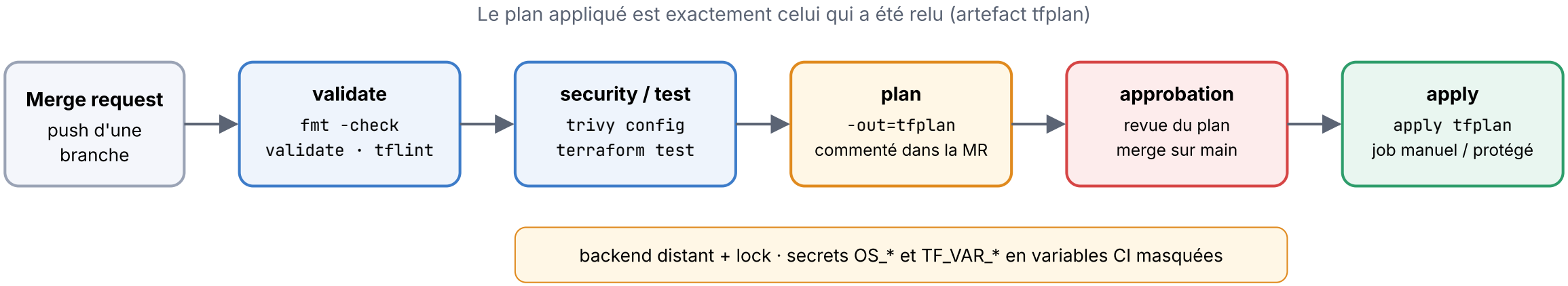
- Créez une machine virtuelle avec Terraform dans un environnement OpenStack.
- Utilisez un provisionneur `remote-exec` pour installer et configurer un serveur web Apache sur la VM.
- Assurez-vous que la configuration réseau permet une connexion SSH sécurisée à la VM pour l'exécution des scripts.
- Documentez les étapes et les commandes utilisées dans votre script Terraform pour référence future.

Utopios® Tous droits réservés

Automatiser son workflow avec Terraform

Automatiser son workflow avec Terraform signifie utiliser Terraform pour codifier et gérer toutes les étapes de votre processus de déploiement d'infrastructure, afin de minimiser l'intervention manuelle, les erreurs et le temps nécessaire pour déployer et gérer votre infrastructure.

1. **Codifier l'Infrastructure :** Écrivez le code qui décrit l'infrastructure que vous souhaitez créer ou modifier. Cela inclut la définition de toutes les ressources dont vous avez besoin (machines virtuelles, réseaux, etc.) en utilisant le langage de configuration de Terraform (HCL).
2. **Gestion du Code Source :** Stockez votre code Terraform dans un système de gestion de version de code source (comme Git) pour suivre les modifications, collaborer avec d'autres membres de l'équipe et gérer les versions de votre code.
3. **Automatisation des Tests :** Écrivez des tests pour votre code Terraform pour vous assurer qu'il fonctionne comme prévu : tests unitaires pour des morceaux de code individuels, tests d'intégration pour s'assurer que différentes parties de votre système fonctionnent ensemble, et tests fonctionnels pour s'assurer que tout le système fonctionne comme prévu.



Utopios® Tous droits réservés

Automatiser son workflow avec Terraform

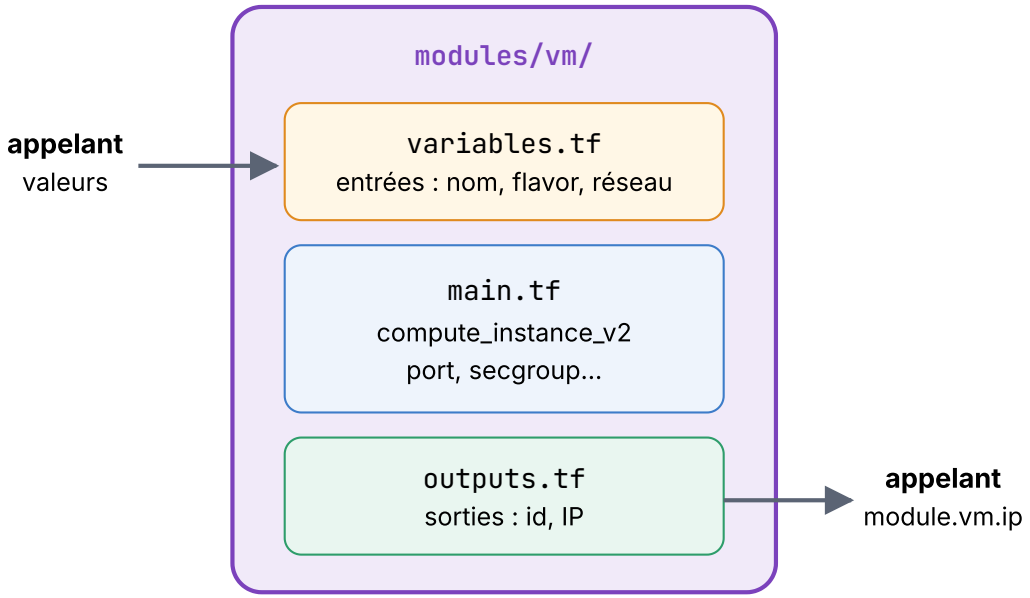
4. **Intégration Continue (CI) :** Utilisez un système d'intégration continue pour automatiser l'exécution de vos tests chaque fois que vous apportez des modifications à votre code. Cela vous aidera à identifier et à corriger rapidement les éventuels problèmes.
5. **Déploiement Continu (CD) :** Utilisez un système de déploiement continu pour automatiser le déploiement de votre code en production. Cela peut inclure l'automatisation de l'exécution de `terraform plan` pour afficher les modifications qui seront apportées à votre infrastructure, et `terraform apply` pour appliquer ces modifications.
6. **Gestion de l'État :** Utilisez un backend distant pour stocker l'état de votre infrastructure. Cela permet à plusieurs membres de l'équipe de travailler sur le même projet en même temps et de gérer l'état de manière sécurisée.
7. **Gestion des Variables :** Utilisez des fichiers de variables et des variables d'environnement pour gérer les configurations spécifiques à chaque environnement. Cela permet de réutiliser le même code pour différents environnements (développement, test, production) en changeant simplement les valeurs des variables.
8. **Modules :** Réutilisez le code en créant des modules pour des morceaux de votre infrastructure que vous utilisez fréquemment. Cela vous permet de ne pas réécrire le même code encore et encore.

Utopios® Tous droits réservés

Les modules Terraform

Les modules Terraform sont utilisés pour encapsuler des groupes de ressources dans des blocs réutilisables. Cela permet de créer des morceaux d'infrastructure réutilisables et partageables, d'organiser l'infrastructure en petits morceaux maniables et de mieux gérer l'infrastructure complexe.

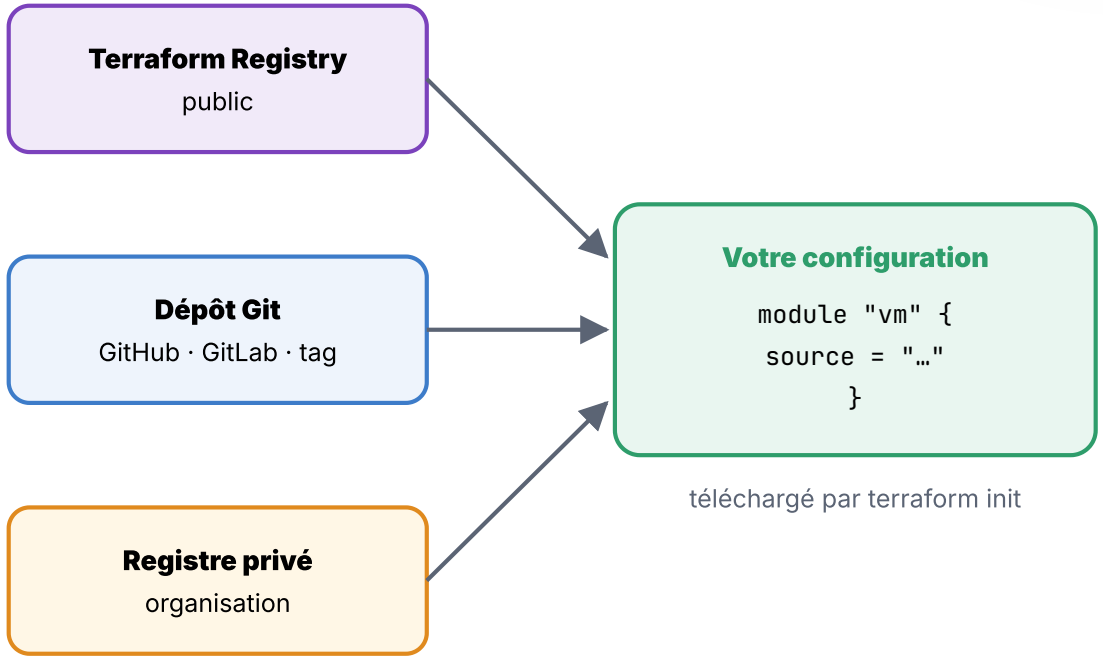
- **1. Création de Modules :** Pour créer un module, vous créez un nouveau répertoire et y placez un fichier de configuration Terraform. Par exemple, si vous avez un ensemble de configurations que vous utilisez fréquemment pour créer des machines virtuelles dans OpenStack, vous pouvez créer un module pour cela.
- Supposons que vous créiez un module pour une machine virtuelle OpenStack. Vous aurez un fichier principal (par exemple, `main.tf`) dans lequel vous définirez les ressources nécessaires pour créer une machine virtuelle. Vous définirez également les variables dans un fichier (par exemple, `variables.tf`) et les sorties dans un autre fichier (par exemple, `outputs.tf`).



Utopios® Tous droits réservés

Les modules Terraform

- **2. Partage de Modules :**
  - Une fois que vous avez créé un module, vous pouvez le partager avec d'autres en le mettant dans un registre de modules comme le Terraform Registry ou un dépôt Git.
  - Si vous travaillez au sein d'une organisation, vous pouvez également utiliser un registre privé.
  - Assurez-vous que le module est bien documenté, de sorte que d'autres sachent comment l'utiliser, quelles sont les variables requises, etc.
- **3. Utilisation de Modules :**
  - Pour utiliser un module, vous l'invoquez dans votre configuration Terraform avec le bloc `module`.
  - Vous spécifiez la source du module (c'est-à-dire l'emplacement où il est stocké, par exemple le Terraform Registry, un dépôt Git, etc.) et les valeurs pour les variables que le module requiert.



Utopios® Tous droits réservés

Les modules Terraform

3. Utilisation de Modules

Terraform Registry

```
module "virtual_machine" {
  source  = "OpenStack/virtual-machine/openstack"
  version = "2.0.0"
  # ...
}
```

Local

```
module "vm" {
  source = "./modules/vm"
  # ...
}
```

Github

```
module "vm" {
  source = "git::https://github.com/your-org/vm-module.git?ref=v1.0.0"

  instance_name = "my-vm"
}
```

Toujours fixer une version : `version` pour un registre, `?ref=` (tag) pour Git.

Utopios® Tous droits réservés

Les modules Terraform - Lab

Objectif : L'objectif de ce TP est de vous apprendre à créer un module Terraform, à le partager sur GitHub et à l'utiliser dans une configuration Terraform pour déployer des ressources dans OpenStack.
Contexte : Vous travaillez en tant qu'ingénieur DevOps pour une entreprise qui utilise intensivement OpenStack. On vous a demandé de créer un module Terraform réutilisable pour déployer des machines virtuelles dans OpenStack. Vous devez ensuite utiliser ce module pour déployer une machine virtuelle dans OpenStack.

Exigences :

1. Créez un nouveau répertoire pour votre module, par exemple `vm-module`. Le module devra créer une machine virtuelle dans OpenStack.
2. Créez un nouveau dépôt sur GitHub et publiez-y votre module. Assurez-vous d'inclure un fichier `README.md` qui explique comment utiliser le module, les variables requises, etc.
3. Créez une nouvelle configuration Terraform dans un répertoire séparé. Dans cette configuration, utilisez le module que vous avez créé pour déployer une machine virtuelle dans OpenStack. Configurez les variables nécessaires pour le module.

Utopios® Tous droits réservés

Gestion des Outputs Terraform pour openstack

- **Introduction :** La gestion des outputs dans Terraform est cruciale pour récupérer des informations essentielles sur les ressources dans des environnements complexes, comme ceux basés sur OpenStack.
- **1. Récupération des Outputs :**
  - Les outputs Terraform permettent de récupérer des valeurs essentielles après l'exécution de la configuration, utiles à la fois dans d'autres parties de votre configuration Terraform et en dehors.
  - Utilisez le bloc `output` pour définir ces valeurs dans votre configuration.

```
output "vm_ip" {
  description = "Adresse IP de la VM"
  value       = openstack_compute_instance_v2.vm.access_ip_v4
}
```

```
$ terraform apply
...
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

vm_ip = "192.168.0.12"
```

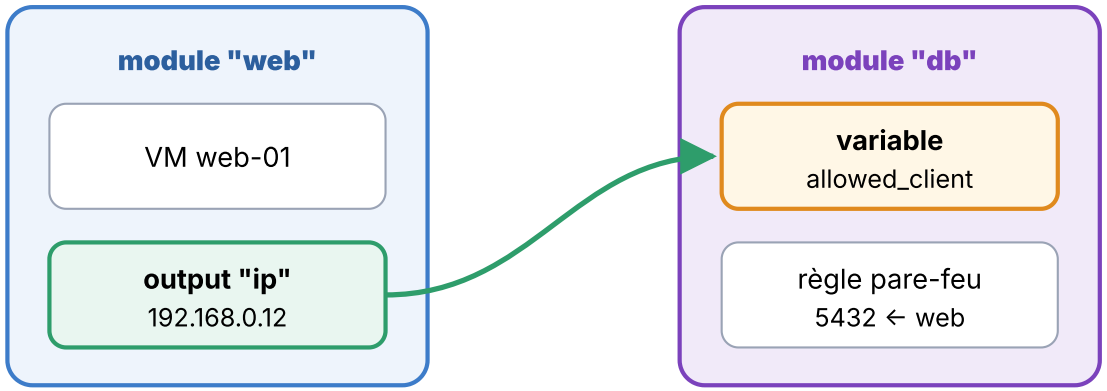
Utopios® Tous droits réservés

Gestion des Outputs dans Terraform pour Openstack

- **2. Utilisation des Outputs dans d'autres modules :**
  - Lors de l'utilisation de modules Terraform, vous pouvez récupérer et exploiter les outputs de ces modules. Par exemple, utiliser l'adresse IP d'une VM OpenStack dans un autre module.

```
module "web" {
  source = "./modules/vm"
  name   = "web-01"
}

module "db" {
  source         = "./modules/db"
  name           = "db-01"
  allowed_client = module.web.ip    # output du module web
}
```



module.web.ip → dépendance implicite web → db

Utopios® Tous droits réservés

Gestion des Outputs dans Terraform pour Openstack

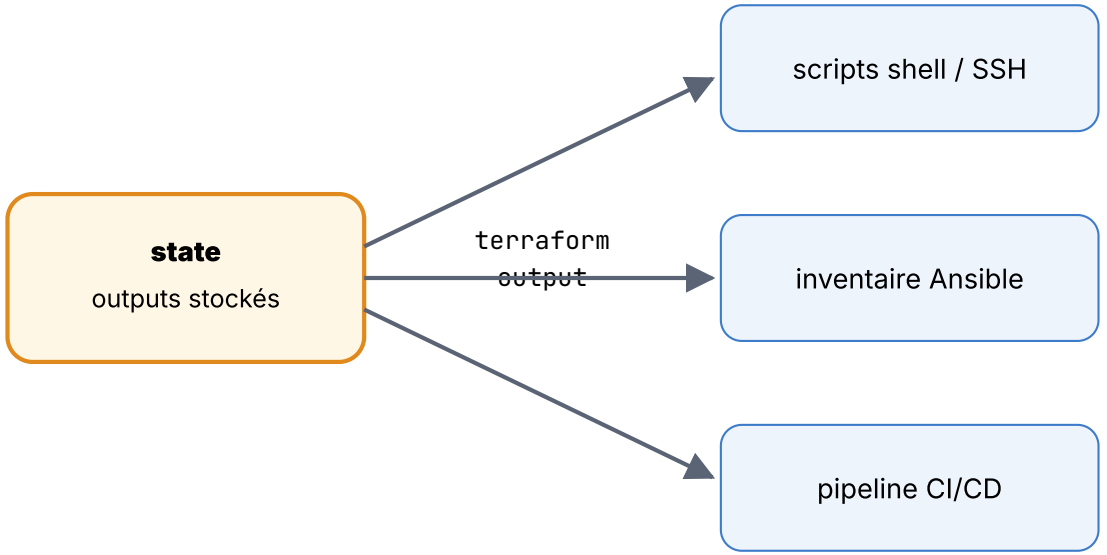
- **3. Utilisation des Outputs en dehors de Terraform :**
  - Vous pouvez récupérer les outputs hors de Terraform avec la commande `terraform output`.
  - Les outputs sensibles, comme les mots de passe, peuvent être marqués comme tels pour empêcher leur affichage.

```
output "db_password" {
  value     = var.db_password
  sensitive = true
}
```

```
$ terraform output
db_password = <sensitive>
vm_ip       = "192.168.0.12"

$ terraform output -raw vm_ip
192.168.0.12

$ ssh ubuntu@$(terraform output -raw vm_ip)
$ terraform output -json   # scripts, Ansible, CI
```



Utopios® Tous droits réservés

Gestion des Outputs Terraform - Exercice Openstack

Objectif : Utiliser Terraform pour déployer une machine virtuelle et un serveur de base de données dans un environnement OpenStack.

Exercice :

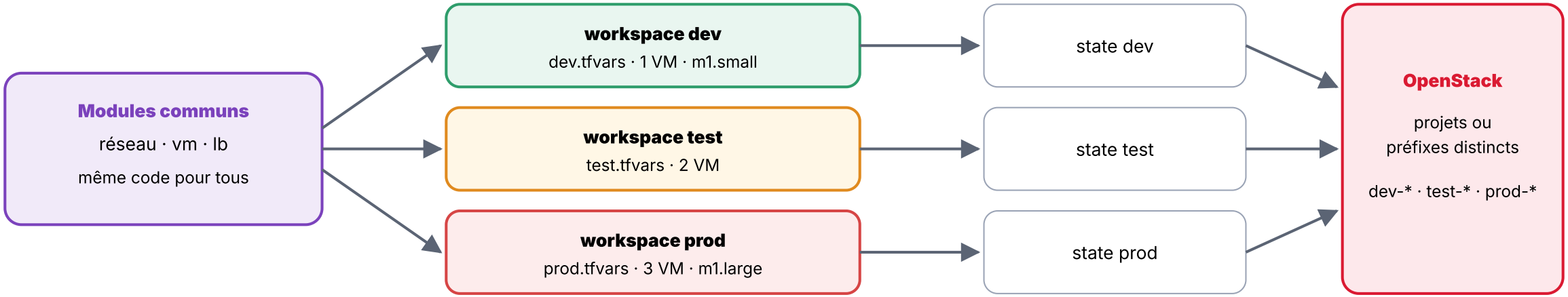
- Déployez une machine virtuelle OpenStack avec Terraform. L'output doit être l'adresse IP de la VM.
- Créez un second module Terraform pour déployer un serveur de base de données sur une VM OpenStack.
  - Le module doit accepter des variables telles que le nom du serveur, l'emplacement, et les identifiants de l'administrateur.
  - Le module crée la VM et installe/configure le serveur de base de données.
- Utilisez l'adresse IP de la VM comme input dans le second module pour configurer des éléments tels que les règles de pare-feu du serveur de base de données.

Utopios® Tous droits réservés

Terraform et les Environnements dans Openstack

Gérer plusieurs environnements dans OpenStack avec Terraform nécessite l'utilisation de configurations, workspaces, variables, et modules pour gérer et déployer des ressources dans des environnements variés tels que développement, production, etc.

1. **Utiliser des Workspaces :** Les workspaces Terraform permettent de gérer différents environnements en isolant leurs états. Par exemple, un workspace pour 'dev' et un pour 'prod', chacun avec son propre état.
2. **Utiliser des Variables :** Les variables servent à définir des valeurs spécifiques à chaque environnement. Par exemple, les configurations de ressources OpenStack peuvent varier entre les environnements de développement et de production.
3. **Utiliser des Modules :** Les modules Terraform regroupent des configurations réutilisables. Un module commun peut être utilisé dans divers environnements avec des variables spécifiques.



Utopios® Tous droits réservés

Terraform et les Environnements Openstack

4 **Gestion des États :** L'état de Terraform doit être stocké de manière sécurisée, accessible depuis différents environnements. Pour OpenStack, vous pouvez utiliser des solutions comme Consul, le stockage objet du cloud (Swift / Ceph via le backend `s3`) ou le state géré par GitLab.

```
terraform {
  backend "consul" {
    address = "consul.example.com"
    path    = "terraform/state"
  }
}
```

5 **Contrôle d'Accès :** Le contrôle d'accès est crucial pour gérer qui peut déployer ou modifier les ressources OpenStack. Utilisez les contrôles d'accès intégrés de OpenStack (projets, rôles Keystone) ou d'autres systèmes de gestion d'identité.

6 **Automatisation :** Automatisez le déploiement de vos configurations Terraform avec des outils de CI/CD comme Jenkins, GitLab CI, ou d'autres pipelines d'intégration continue.

Utopios® Tous droits réservés

Terraform et les Environnements - Workspaces Openstack

Terraform offre la possibilité de gérer différents environnements avec des « workspaces ».

1. **Créer un Workspace :** Créez un nouveau « workspace » pour chaque environnement (par exemple, développement, test, production).

```
terraform workspace new development
```

2. **Sélectionner un Workspace :** Sélectionnez le « workspace » approprié pour déployer ou modifier les ressources OpenStack.

```
terraform workspace select development
```

```
backend local

.
├── main.tf
├── terraform.tfstate # default
└── terraform.tfstate.d/
├── development/
│ └── terraform.tfstate
└── production/
└── terraform.tfstate

un même code, un state par workspace · ${terraform.workspace}
```

Utopios® Tous droits réservés

Terraform et les Environnements - Workspaces Openstack

3 **Lister les Workspaces :** Visualisez tous les « workspaces » existants pour gérer les différents environnements.

```
terraform workspace list
```

4 **Supprimer un Workspace :** Supprimez un « workspace » lorsque son utilisation n'est plus nécessaire.

```
terraform workspace delete development
```

```
$ terraform workspace list
  default
* development
  production
```

utiliser le workspace dans le code

```
resource "openstack_compute_instance_v2" "web" {
  name = "web-${terraform.workspace}"
  # web-development, web-production
}
```

On ne peut supprimer qu'un workspace **vide** (ressources détruites) et non sélectionné.

Utopios® Tous droits réservés

Terraform et les Environnements - Workspaces - Lab Openstack

Objectif : Apprenez à gérer différents environnements comme le développement, le test et la production dans OpenStack en utilisant les workspaces de Terraform.

Exercice :

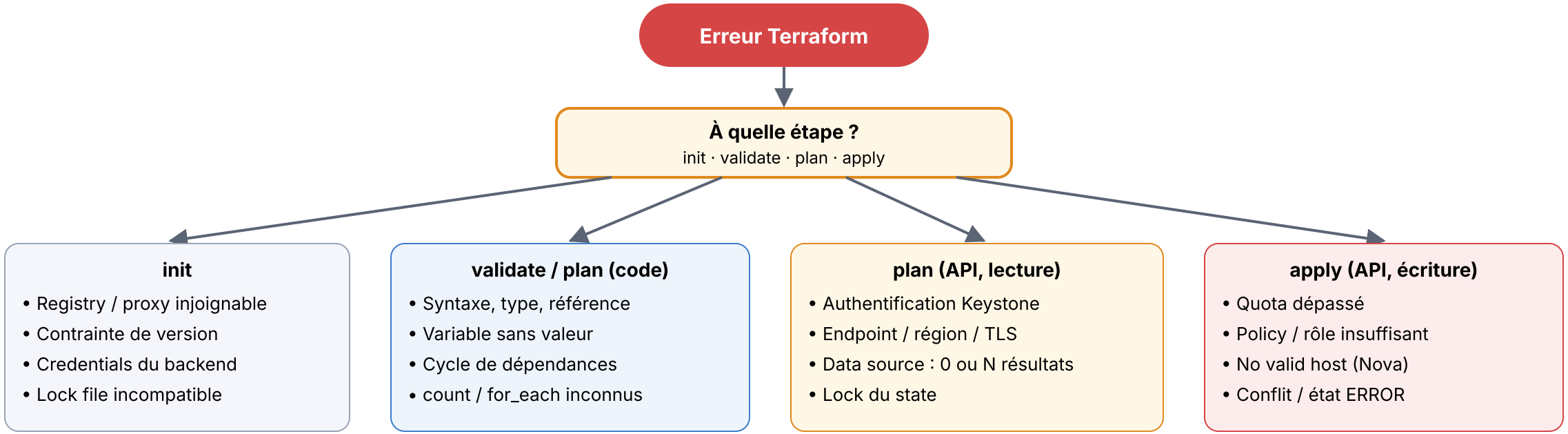
- Configurez les credentials OpenStack pour Terraform.
- Configurez le backend de Terraform pour stocker l'état dans une solution compatible avec OpenStack.
- Créez un fichier de variables pour vos configurations.
- Définissez les ressources OpenStack dans un fichier de configuration Terraform.
- Utilisez les workspaces de Terraform pour créer et gérer différents environnements.

Utopios® Tous droits réservés

Troubleshooting

Le dépannage dans Terraform peut impliquer plusieurs étapes et techniques différentes en fonction de la nature du problème.

- **Consultez les journaux et les messages d'erreur :** Les messages d'erreur affichés par Terraform peuvent souvent fournir des informations précieuses sur ce qui ne va pas. Assurez-vous de lire et de comprendre les messages d'erreur.
- **Utilisez terraform plan :** Avant d'appliquer les changements, utilisez toujours la commande `terraform plan` pour voir quels seront les changements apportés à votre infrastructure.
- **Utilisez la commande terraform show :** Cette commande vous permet de visualiser l'état actuel de votre infrastructure et peut être utile pour identifier les problèmes.
- **Vérifiez l'état de Terraform :** Utilisez la commande `terraform state list` pour voir toutes les ressources dans l'état actuel de Terraform. Cela peut être utile pour vérifier si une ressource a été créée, modifiée ou détruite.



Utopios® Tous droits réservés

Troubleshooting

- **Vérifiez les versions de Terraform et des providers :** Assurez-vous d'utiliser des versions de Terraform et des providers compatibles. Vous pouvez spécifier les versions dans votre fichier de configuration Terraform.
- **Vérifiez les dépendances des ressources :** Assurez-vous que les ressources sont créées dans le bon ordre et que les dépendances sont correctement configurées. Terraform gère généralement les dépendances automatiquement, mais dans certains cas, vous devrez peut-être les spécifier explicitement à l'aide de l'argument `depends_on`.
- **Vérifiez les permissions et les politiques :** Assurez-vous que le compte utilisé par Terraform a les permissions nécessaires (rôle Keystone sur le projet) pour créer, modifier ou détruire les ressources.
- **Vérifiez les limites de votre compte :** OpenStack applique des quotas sur le nombre de ressources que vous pouvez créer (`openstack quota show`). Assurez-vous que vous n'avez pas atteint ces limites.
- **Consultez la documentation :** La documentation de Terraform et des providers peut souvent fournir des informations utiles pour résoudre les problèmes.

Utopios® Tous droits réservés

Troubleshooting

- **Vérifiez les valeurs des variables :** Assurez-vous que les valeurs des variables que vous avez spécifiées sont correctes. Vous pouvez utiliser la commande `terraform console` pour vérifier les valeurs des variables et tester les expressions Terraform.
- **Utilisez terraform validate :** Cette commande vérifie que votre fichier de configuration est syntaxiquement correct et valide.
- **Vérifiez les ressources créées manuellement :** Assurez-vous que les ressources que vous gérez avec Terraform n'ont pas été modifiées manuellement ou par un autre outil. Terraform peut avoir du mal à gérer les ressources qui ne sont pas entièrement gérées par lui-même.
- **Utilisez des sorties de débogage :** Vous pouvez obtenir des informations supplémentaires sur l'exécution de Terraform en définissant la variable d'environnement `TF_LOG` à `DEBUG` ou `TRACE`.
- **Utilisez le terraform graph :** Cette commande génère un graphique visuel de vos ressources Terraform et de leurs dépendances. Vous pouvez utiliser un outil comme Graphviz pour visualiser ce graphique.

```
terraform console
> var.flavor
"m1.small"

terraform validate
Success! The configuration is valid.

export TF_LOG=DEBUG
export TF_LOG_PATH=./terraform.log
export OS_DEBUG=1      # requêtes API OpenStack
terraform plan

terraform graph | dot -Tsvg > graph.svg
```

Utopios® Tous droits réservés